

Tabla de Contenidos

1	Framework de simulación de redes vehiculares	1
1.1	Introducción	1
1.1.1	Instalación del framework	2
1.2	Instalación del framework en contenedores Docker	2
1.2.1	Instalación local de OMNET++ y Veins	3
1.2.2	Simulación de redes vehiculares en OMNET++	8
1.3	Desarrollo	9
2	Introducción al Simulador de movilidad Urbana (SUMO)	11
2.1	Objetivos	11
2.2	Introducción	11
2.3	Materiales	12
2.4	Instrucciones	12
2.4.1	Parte I: Instalación de SUMO	12
2.4.2	Instalación en Contenedores Docker	14
2.5	Instalación del framework en contenedores Docker	14
2.5.1	Parte II: Generación de archivos de simulación de SUMO	15
2.5.2	Parte III: Generación de una simulación usando las consolas CLI y GUI de SUMO	21
2.5.3	Parte IV: osmWebWizard	22

3	Demanda de movilidad vehicular	25
3.1	Objetivos	25
3.2	Introducción	25
3.3	Modelos de asignación de tráfico	25
3.4	Herramientas de generación de demanda vehicular	26
3.5	Desarrollo	26
3.5.1	Parte I: Flujos Aleatorios con RandomTrips.py	27
3.5.2	Parte II: Demanda basada en el comportamiento (TAZ y matrices OD)	27
3.5.3	Parte III: Enrutamiento dinámico duarouter	31
3.5.4	Parte IV: Análisis de modelos de generación de demanda vehicular	32
4	Estadísticas de movilidad vehicular en SUMO	35
4.1	Objetivos	35
4.2	Introducción	35
4.3	Instrucciones	36
4.3.1	Parte I: Generar archivo de salida (outputs) en formato .csv	36
4.3.2	Parte II: Generar gráficas generales de visualización.	37
4.3.3	Parte III: Generar gráficas usando herramientas de visualización de SUMO.	38
4.3.4	Trayectorias:	38
4.3.5	Velocidad:	39
4.3.6	Densidad de vehículos:	40
4.3.7	Emisiones de gases:	41
5	Introducción a TraCI (Traffic Control Interface)	43
5.1	Objetivos	43
5.2	Introducción	43
5.3	Desarrollo	45
5.3.1	Parte I: Instalar librería TraCI para Python.	46
5.3.2	Parte II: Ejecutar una simulación desde Python usando la interfaz TraCI	46
5.3.3	Parte III: Ejemplos de interacción TraCI - SUMO.	48
5.4	Reto TraCI	51
6	Sistemas inteligentes de transporte ITS basados en ML	52
6.1	Objetivos	52
6.2	Introducción	52
6.3	Materiales	53

6.4	Instrucciones	54
6.4.1	Aprendizaje Supervisado	54
6.4.2	Aprendizaje No Supervisado	56
6.4.3	Aprendizaje de maquina aplicado a ITS	57
6.5	Objetivos	57
6.6	Introducción	57
6.7	Materiales	58
6.8	Instrucciones	58
6.8.1	Aprendizaje por refuerzo	60
6.9	Objetivos	60
6.10	Introducción	60
6.11	Materiales	62
6.12	Instrucciones	62
7	Proyectos ITSs.	74
7.1	Objetivos	74
7.2	Temas propuestos	74
7.2.1	Tema 1: (Path planning de vuelo)	74
7.2.2	Tema 2: (Autonomía: Energía de drones con carga)	74
7.2.3	Tema 3: (Lora/ESPNow/Wifi)	74
7.2.4	Tema 1: (FANET) - Tiene muchas limitaciones energia/camara	75
7.2.5	Tema 2: (SUMO) - PROYECTO DE VINCULACION SE puede agregar ahora	75
7.2.6	Tema 4: (CANBUS)	75
7.2.7	Tema 5: (Redes Vehiculares)	75
7.2.8	Tema 6: (ITS) Detección de señales de tránsito de Cuenca - limites de velocidad	75
7.2.9	Tema 7: (SUMO)	76
7.2.10	Tema 8: (SUMO)	76
8	Implementación de una red CAN 1	77
8.1	Objetivos	77
8.2	Introducción	77
8.3	Materiales	78
8.4	Instrucciones	78
8.4.1	Parte I: Red CAN x nodos	78
8.4.2	Parte II: Análisis de tramas CAN	80

9	Simulación de una red CAN 2	82
9.1	Objetivos	82
9.2	Introducción	82
9.3	Materiales	83
9.4	Instrucciones	83
9.4.1	Parte I: Red CAN Virtual en Linux	83
9.4.2	Parte II: Análisis de las tramas CAN con SavvyCAN	84
9.4.3	Parte III: Hack CAN Bus con SavvyCAN	86
10	Framework de simulación de redes vehiculares	88
10.1	Introducción	88
10.1.1	Instalación del framework	89
10.2	Instalación del framework en contenedores Docker	89
10.2.1	Instalación local de OMNET++ y Veins	90
10.2.2	Simulación de redes vehiculares en OMNET++	95
10.3	Desarrollo	96
11	Veins: IEEE WAVE messages BSM/WSM (WSMP)	98
11.1	Introducción	98
11.1.1	Comparación entre módulos TraCIDemo11p.cc y TraCIDemo11RSUp.cc	98
11.1.2	Comparación entre mensajes BaseFrame1609_4 y DemoBaseFrame	99
11.2	Funciones de TraCIDemo11p.cc	99
11.3	Desarrollo	101
11.3.1	Parte IV: Análisis de modelos de generación de demanda vehicular	103
12	Veins: Implementar e investigar el greedy forwarding	104
12.1	Introducción	104
12.1.1	Comparación entre módulos TraCIDemo11p.cc y TraCIDemo11RSUp.cc	104
12.1.2	Comparación entre mensajes BaseFrame1609_4 y DemoBaseFrame	105
12.2	Funciones de TraCIDemo11p.cc	105
12.3	Desarrollo	107
12.3.1	Parte IV: Análisis de modelos de generación de demanda vehicular	109
13	Movilidad Vehicular: Ejercicio en Clase	110
13.1	Parte I Generación del Escenario	110
13.2	Escenarios de Simulación	110

13.3	Parte II Outputs y Estadísticas	111
13.4	Parte III Ideas para Algoritmos de Machine Learning	112
13.4.1	Aprendizaje Supervisado	112
13.4.2	Aprendizaje No Supervisado	112
13.4.3	Aprendizaje por Refuerzo	112
13.5	Entregables del Reto	113
13.6	Criterio Diferenciador	113
	References	114

Introducción

Instalación del framework

Instalación del framework en contenedores Docker

Instalación local de OMNET++ y Veins

Simulación de redes vehiculares en OMNET++

Desarrollo



1. Framework de simulación de redes vehiculares

En esta práctica, el estudiante explora el entorno de VEINS

Objetivos

- Implementar una red vehicular en Docker.
- Familiarizarse con el framework de simulación OMNET++, Veins y SUMO.
- Actualizar su mapa de Cuenca en el escenario de simulación.

1.1 Introducción

En la presente práctica, nos enfocamos en instalar el framework de simulación y familiarizarnos con el proceso de ejecución de una simulación de red vehicular de corto alcance o DSRC (Dedicated Short Range Communications).

La presente práctica tiene como objetivo la instalación del entorno de simulación y el reconocimiento del flujo de ejecución para redes vehiculares de corto alcance, basadas en comunicaciones **DSRC (Dedicated Short Range Communications)**. Para ello, se integran herramientas que permiten modelar tanto la movilidad física como las capas de red y sistemas de transportación definidos como aplicación.

En prácticas anteriores adquirimos las destrezas necesarias para modelar la movilidad vehicular usando SUMO, un simulador de naturaleza microscópica que permite modelar la dinámica de cada vehículo de forma individual. En esta práctica introducimos el orquestador de las simulaciones llamado **OMNET++**. OMNET++ es un simulador de eventos discretos, modular y extensible basado en componentes de C++. Su arquitectura permite ensamblar módulos mediante el lenguaje de alto nivel **NED**, facilitando la reutilización de código y la creación de redes jerárquicas complejas. Al integrarse con un IDE basado en Eclipse, OMNeT++ proporciona las herramientas necesarias para la emulación de protocolos y la simulación de eventos discretos, constituyendo la base sobre la cual se despliegan las funcionalidades de comunicación de las redes vehiculares.

Por otra parte, tenemos a **Veins** (The vehicular network simulation framework), que se posiciona como una herramienta de referencia al sustentar sus simulaciones en modelos de movilidad vehicular de alta fidelidad, desarrollados y validados por la comunidad científica de transporte y tráfico. En el ámbito de las comunicaciones, Veins destaca por la implementación de las capas física **IEEE 802.11p** y capas superiores definidas en **IEEE 1609.4 DSRC/WAVE**. Estas incluyen características críticas como la operación multicanal, el acceso al canal con calidad de servicio

(*QoS*), y el modelado realista de efectos de ruido e interferencias, garantizando que el análisis de la red sea tan preciso como el de la movilidad física.

Un componente crítico en este ecosistema es el protocolo **TraCI** (*Traffic Control Interface*), el cual actúa como el motor de sincronización bidireccional. A través de una conexión TCP, **OMNeT++** asume el rol de cliente, enviando comandos a **SUMO** (el servidor) para recuperar datos de movilidad o alterar el comportamiento de los vehículos en tiempo real. Esta interacción permite, por ejemplo, que un mensaje de seguridad transmitido en la red inalámbrica provoque un cambio de ruta o una frenada de emergencia en el simulador de tráfico.

Finalmente, para simplificar el despliegue de estas herramientas y evitar conflictos de dependencias entre versiones de bibliotecas, se hace uso de contenedores **Docker**. Esta metodología de virtualización ligera encapsula todo el entorno de ejecución (**SUMO**, **OMNeT++** y **Veins**), permitiendo una configuración rápida y reproducible.

1.1.1 Instalación del framework

El despliegue del entorno de simulación (**SUMO**, **OMNeT++** y **Veins**) puede realizarse mediante tres enfoques principales, cada uno con distintas implicaciones en términos de rendimiento y facilidad de configuración:

- **Contenedores (Docker):** Es la opción recomendada para entornos académicos. Consiste en utilizar una imagen preconfigurada que encapsula todas las dependencias y versiones compatibles de las herramientas. Su principal ventaja es la reproducibilidad, ya que elimina conflictos con las librerías del sistema operativo anfitrión y permite un despliegue inmediato.
- **Instalación Nativa:** Implica la compilación manual de cada componente sobre el sistema operativo (generalmente distribuciones GNU/Linux). Requiere la gestión directa de dependencias como el compilador *gcc*, librerías gráficas *Qt* y entornos de ejecución de *Python*. Aunque ofrece el máximo rendimiento de hardware, es proclive a errores de compatibilidad entre versiones.
- **Máquinas Virtuales (VM):** Consiste en el uso de un sistema operativo completo (comúnmente una imagen *.ova*) que ya incluye el software instalado. Es la alternativa más sencilla aunque presenta una sobrecarga (*overhead*) considerable en el uso de memoria RAM y CPU en comparación con las opciones anteriores.

1.2 Instalación del framework en contenedores Docker

Para facilitar el despliegue de **OMNeT++**, **Veins** y **SUMO** para simulación de redes vehiculares, se ha puesto a disposición dos opciones para los estudiantes:

1. **Dockerfile:** el repositorio en GitHub (<https://github.com/Pbarbecho/VANETS.git>) contiene el archivo *Dockerfile* necesario para construir la imagen del entorno. Requiere de unos minutos para descargar y compilar las aplicaciones. El estudiante ha de construir el contenedor en base al *dockerfile* compartido:

```
docker build -t oppveinsumo:latest .
```

2. **Docker Image:** puede descargar directamente la imagen compartida en **DockerHub**. Considere que la imagen pesa alrededor de 5GB.

```
docker pull pbarbecho/vanets
# Utilice la plataforma adecuada para su PC
# --platform string    Set platform if server is multi-platform
# capable
# docker pull --platform linux/arm64 pbarbecho/vanets
```

En ambos casos se utiliza como base **Ubuntu 22.04** y se destacan las siguientes configuraciones para garantizar la compatibilidad entre OMNeT++ 6.3.0, SUMO 1.12 y Veins 5.3.1 (<https://veins.car2x.org/download/#compatibility>):

1. **Dependencias del Sistema y Gráficas:** Se instalan herramientas de compilación (*clang*, *make*, *build-essential*), librerías para el manejo de XML y compresión, y el stack completo de **Qt6** junto con bibliotecas de renderizado (*Mesa*, *OpenGL*, *OpenSceneGraph*). Esto es fundamental para que la interfaz gráfica de OMNeT++ (QtEnv) y SUMO-GUI funcionen correctamente mediante redirección X11. Además, se instalan directamente los binarios de **SUMO** y sus herramientas asociadas (*netconvert*, *netedit*, etc.).
2. **Ecosistema Python:** Se configuran los módulos necesarios para el análisis de resultados en OMNeT++, incluyendo *Matplotlib*, *NumPy*, *Pandas* y *SciPy*, asegurando que el entorno de análisis estadístico esté listo para su uso.
3. **Compatibilidad de Escritorio:** Se implementan *scripts* de sustitución para las herramientas *xdg-utils*. Esto evita errores comunes en entornos de contenedores donde no existe un gestor de menús de escritorio real, permitiendo que el proceso de instalación de OMNeT++ finalice sin interrupciones.
4. **Variables de Entorno:** Se definen rutas críticas como SUMO_HOME y se configuran parámetros de estabilidad gráfica como LIBGL_ALWAYS_SOFTWARE=1, lo cual fuerza el renderizado por software para evitar problemas de compatibilidad con drivers de video específicos del host.

Nota de Instalación

Para ejecutar aplicaciones gráficas desde el contenedor, asegúrese de tener instalado un servidor X (como Xming o VcXsrv en Windows, o XQuartz en macOS) y configurada correctamente la variable de entorno DISPLAY.

1.2.1 Instalación local de OMNeT++ y Veins

Una de las decisiones de diseño clave en esta práctica es la instalación de **OMNeT++** en un directorio compartido entre la máquina anfitrión y el contenedor Docker. A diferencia de las librerías del sistema que residen internamente en la imagen, como SUMO, el entorno de simulación se ubica en un volumen persistente. Esta configuración ofrece tres ventajas críticas para el flujo de trabajo:

- **Persistencia de Datos:** Evita la pérdida de configuraciones, archivos de resultados (*.sca*, *.vec*) y binarios compilados al detener o eliminar el contenedor.
- **Desarrollo Híbrido:** Permite al usuario editar el código fuente de los protocolos o archivos *NED* utilizando un IDE local (como VS Code o Eclipse, ver Fig. 10.1) en el host, mientras que la compilación y ejecución se realizan dentro del entorno controlado del contenedor. Se

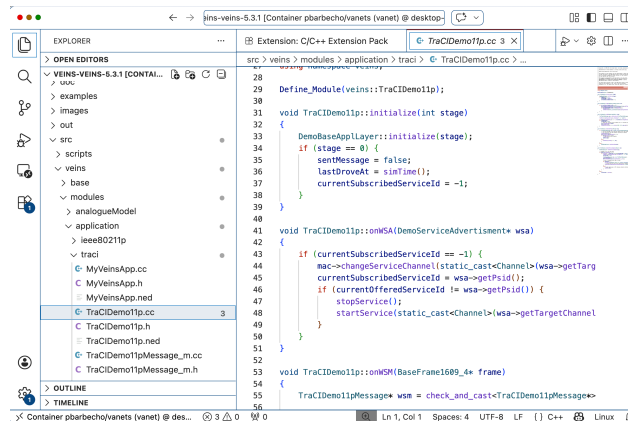


Figure 1.1: Codificación en VS Code usando la extensión Dev Containers

recomienda instalar la extensión "Dev Containers", y selecciona agregar el contenedor en ejecución "Attach to Running Container" para navegar y cargar el código de Veins.

- **Optimización de Espacio:** Al separar el framework del sistema operativo base del contenedor, se facilita la actualización de la imagen sin necesidad de reinstalar o volver a descargar los proyectos de simulación y las librerías de **Veins**.

Tras la construcción exitosa de la imagen (por ejemplo, `vanetimg`), el siguiente paso consiste en instanciar el contenedor para proceder con la instalación y compilación local de **OMNeT++** y **Veins**. Este procedimiento garantiza que el núcleo del simulador resida en un volumen persistente, facilitando futuras modificaciones sin alterar la imagen base.

A continuación, se detallan los pasos necesarios para una máquina anfitrión con sistema operativo **Linux**:

1. Lo primero será crear un directorio compartido en la maquina local donde se instalarán OM-NET++ y Veins.

```
mkdir vanet && cd vanet
```

2. VcXsrv: En el caso que de usar el sistema operativo WINDOWS, antes de lanzar el contenedor es necesario instalar un sistema de ventanas o X Server. Se sugiere usar el VcXsrv de código abierto que permite que las aplicaciones de Linux (que funcionan dentro de Docker o WSL) lancen sus ventanas o menús en su escritorio de Windows. En nuestro caso necesitamos la interfaz del IDE de OMNeT++ y SUMO-GUI.

- (a) VcXsrv se puede descargar de: <https://sourceforge.net/projects/vcxsrv/files/latest/download>
- (b) Una vez descargado el .exe, instalarlo. La primera vez en ejecutar el programa ha de configurar:
 - Pantalla 1: Seleccione Multiple windows (deja el Display number en -1).
 - Pantalla 2: Seleccione Start no client.
 - Pantalla 3: *Marcar la casilla Disable access control.

- (c) Ahora que el programa ya está "escuchando" en Windows, regresar dentro del contenedor y ejecutar, `export DISPLAY=host.docker.internal:0.0`
3. Lanzamos el contenedor dentro del directorio compartido donde se instalan OMNET++ y Veins. Note que para el ejemplo el nombre de la imagen es `oppveinsumo`. Se requiere de alrededor de 400 MB de espacio en disco:

```
docker run -it --rm --name vanet -e DISPLAY=host.docker.internal:0 -v /tmp/.X11-unix:/tmp/.X11-unix -v "$(pwd):/root/omnet" oppveinsumo
```

- `-rm`: Indica a Docker que debe eliminar el contenedor automáticamente al cerrar la sesión. La persistencia se maneja mediante el volumen compartido.
 - `-name vanet`: nombre del contenedor en ejecución para facilitar su identificación. Dado que necesitamos ejecutar instancias simultáneas, es importante definir un nombre para el contenedor.
 - `-e DISPLAY=host.docker.internal:0`: Define una variable de entorno que indica a las aplicaciones gráficas (como el IDE de OMNET++ o la interfaz gráfica de SUMO) hacia dónde enviar la salida de video. En Windows/macOS, apunta al servidor X local. Verifique que su servidor local esté activo.
 - `-v /tmp/.X11-unix:/tmp/.X11-unix`: Monta el socket de comunicación del sistema de ventanas X11. Es esencial en sistemas Linux para que el contenedor pueda comunicarse directamente con el servidor gráfico del anfitrión. En windows no es necesario.
 - `-v "$(pwd):/root/omnet"`: Crea el volumen compartido entre el directorio actual de su terminal (e.j. `vanet`) y la carpeta de trabajo del contenedor. Como OMNeT++ se instala en esta ruta compartida, todos los cambios en el código y los archivos de resultados se guardarán físicamente en su computadora de manera local, no dentro del contenedor.
 - `oppveinsumo`: Especifica el nombre de la imagen que nos descargamos previamente o construimos a partir del `dockerfile`.
4. Dentro del contenedor, el control de las herramientas de **OMNeT++** se centraliza en la utilidad `opp_env`, que permite inicializar el espacio de trabajo mediante el parámetro `init`. Luego, para instalar el entorno de OMNET++, dentro del contenedor usamos `install` seguido de la versión de OMNET++:

```
opp_env init --nixless -workspace  
opp_env install omnetpp-6.3.0
```

5. Es importante destacar que, debido a que el *framework* reside en un directorio compartido y no en las rutas globales del sistema, la ejecución directa del comando `omnetpp` resultará inicialmente en un error de *"comando no encontrado"*. Para resolver esto y mapear correctamente los binarios instalados, es imperativo ejecutar `opp_env shell`; este comando activa las variables de entorno necesarias y carga las librerías dinámicas, permitiendo que la terminal reconozca y ejecute todas las herramientas de gestión del simulador.

```
opp_env shell
```

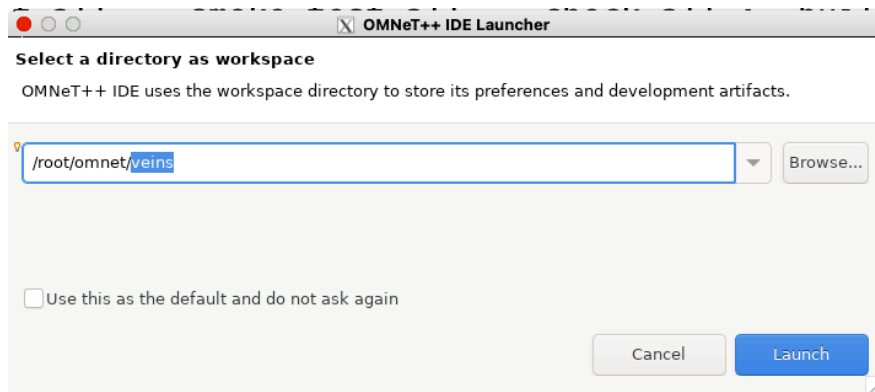


Figure 1.2: Veins path. Note que hacer referencia al directorio compartido con la máquina anfitriona.

En este punto note que el prompt cambia, indicando que ya podemos ejecutar comandos de OMNET++:

```
omnetpp-6.3.0:~/omnet/omnetpp-6.3.0$.
```

6. Ejecutamos omnetpp que lanza la interfaz gráfica del IDE, a través de la cuál cargamos y compilamos Veins:

```
omnetpp
```

Agregue el directorio de veins al final del path (ver Fig. 10.2), donde guardaremos el código de veins. De esta forma puede conectar cualquier IDE de programación con soporte para C++, para modificar el código de veins que contiene la definición de la red vehicular.

7. Ahora vamos a cargar el proyecto de Veins en OMNET++. Para esto, nos descargamos Veins 5.3.1 de <https://veins.car2x.org/download/> y copiamos la carpeta a la ubicación compartida con el docker para sea accesible desde este.
8. Desde el entorno de OMNET++, nos dirigimos a *File* → *Open Projects from file system* y en *Import Source* buscamos la carpeta del proyecto de Veins (ver Fig. 10.3).
9. Compilamos el proyecto con *Ctrl + B*. Puede tardar unos minutos (ver Fig. 10.4). En la ventana de *Project explorer*, nos dirigimos a *examples* → *veins* → *omnet.ini* y le damos a la pestaña *Source* para ver el archivo de configuración .ini en modo texto.
10. Ejecución del servidor SUMO. Para la evaluación de redes vehiculares (VANETs), es imprescindible contar con un entorno que modele de forma realista la movilidad de los nodos. En esta arquitectura, el simulador **SUMO** opera en modo servidor, delegando el control de la simulación a **Veins**, que actúa como el nodo cliente. Esta interacción se realiza mediante el protocolo **TraCI** (*Traffic Control Interface*), el cual permite una sincronización bidireccional: mientras SUMO gestiona la dinámica física de los vehículos, Veins inyecta la lógica de red y puede alterar el comportamiento del tráfico en tiempo real. Para habilitar esta comunicación, es necesario ejecutar un *script* de enlace que ponga a SUMO en estado de escucha (*listening mode*) en un puerto TCP específico (comúnmente el 9999).

1.2 Instalación del framework en contenedores Docker

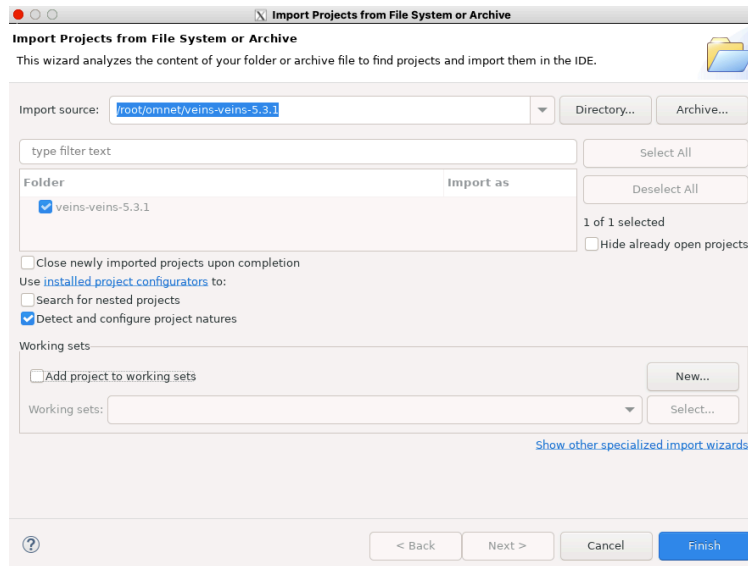


Figure 1.3: Import Veins project en OMNET++

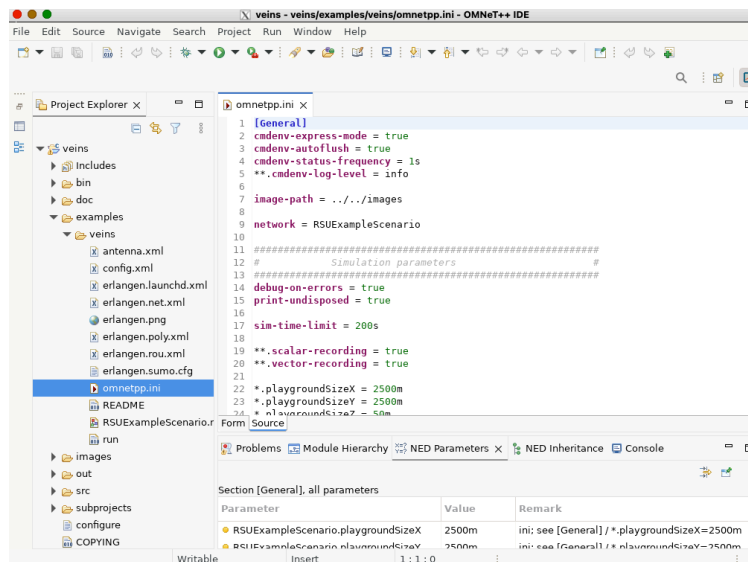


Figure 1.4: Archivo *omnet.ini* del Veins project

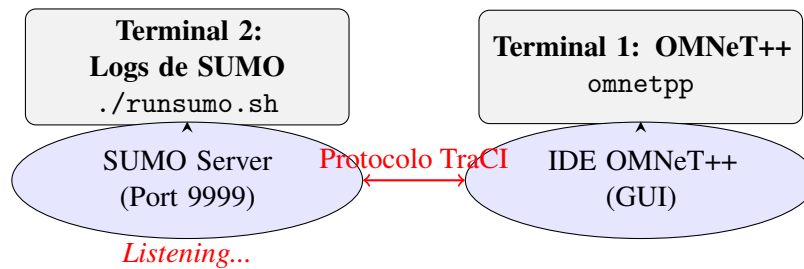


Figure 1.5: Interacción multi-terminal para simulación de redes vehiculares.

Un ejemplo del shell script `run_sumo.sh` se encuentra disponible en el <https://github.com/Pbarbecho/VANETS.git>, pero depende de su configuración de directorios en el contenedor. Cree el shel script en el directorio compartido para que esté disponible desde el contenedor y asigne permisos de ejecución:

```
# Comando para lanzar el proxy de escucha de SUMO
/root/omnet/veins-veins-5.3.1/sumo-launchd.py -vv -c /usr/share/sumo/bin/sumo
```

Nota sobre la Interfaz TraCI

Sin la ejecución previa de este servidor de escucha, OMNeT++ no podrá establecer el túnel de datos con SUMO, lo que resultará en un error de conexión (*Connection refused*) al intentar iniciar cualquier escenario de simulación vehicular.

1.2.2 Simulación de redes vehiculares en OMNET++

Para el correcto funcionamiento de la simulación, es necesario mantener un flujo de trabajo síncrono dividido en dos procesos independientes que se comunican internamente. La Fig. 10.5 muestra la orquestación entre el servidor de movilidad y el entorno de desarrollo:

- **Terminal 2 (Servidor):** Utilice el comando `docker exec` para lanzar una nueva terminal donde ejecutará el script `runsumo.sh` (que invoca a `sumo-launchd.py`). Este proceso debe permanecer abierto y en estado "*Listening on port 9999*" para actuar como puente de movilidad.

```
# Comando para verificar el nombre del contenedor activo
docker ps

# Comando para lanzar otra terminal de un contenedor activo.
docker exec -it [NOMBRE CONTENEDOR] bash
```

- **Terminal 1 (Cliente):** Lanza la interfaz gráfica de OMNeT++. Desde el IDE, al iniciar la simulación, se establece una conexión TCP con la Terminal 2 para sincronizar el movimiento de los vehículos en el escenario de red.

En este punto basta con ejecutar la simulación desde el IDE de OMNeT++. Note que OMNeT++ detectará únicamente como archivo ejecutable el archivo `omnet.ini` (ver Fig. 10.4). Ubi-

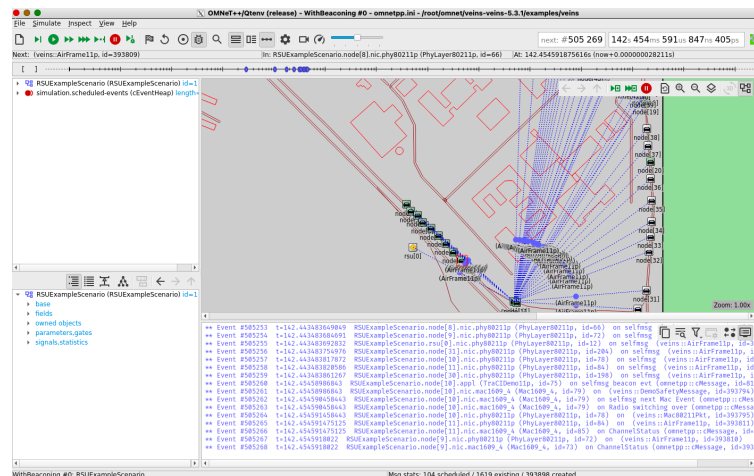


Figure 1.6: OMNET++ (Qtev) simulations environment

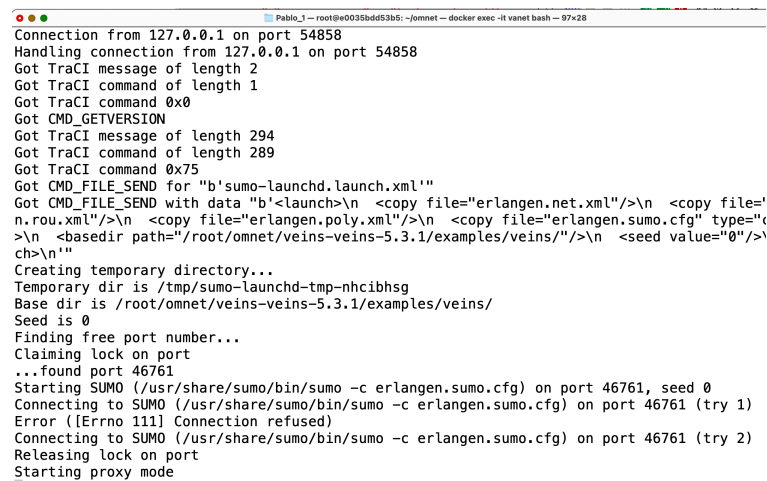


Figure 1.7: SUMO Logs

cados sobre el archivo `omnet.ini`, le damos al ícono de play en el IDE de OMNET++. Se abrirá una nueva interfaz gráfica (Qtenv) 10.6 que nos permite visualizar la simulación de la red vehicular (ver Fig. 10.6).

Por otra parte, podemos referirnos a la Terminal 2 para verificar el avance de la simulación en SUMO (logs), (ver Fig. 10.7)

1.3 Desarrollo

Valide con el docente

El desarrollo de la práctica se divide en dos etapas secuenciales. Una vez completadas, es obligatoria la validación de los resultados con el docente. **La presente práctica no requiere informe.**

1. **Parte I:** Ejecutar la simulación de ejemplo incorporada en el proyecto de Veins .
2. **Parte IV:** Reemplazar los archivos de SUMO (.net, .rou, .poly, sumo.cfg) que vienen por defecto en Veins. Utilice sus archivos de SUMO y el mapa de Cuenca. Realice los ajustes necesarios en el archivo `omnet.ini` para que la simulación se ejecute satisfactoriamente. Note que veins incorpora un archivo adicional llamado `earlang.launchd.xml` que define la ubicación de los archivos de configuración de SUMO. Realice los siguientes ajustes:
 - Primero, defina el espacio de simulación (Playground) de acuerdo con su mapa.
 - Dentro de este mapa, posicione estratégicamente la RSU (Road Side Unit) en las coordenadas (x,y,z) .
 - Programe el escenario de accidente vinculándolo a un vehículo específico (por ejemplo, el `node[5]`), configurando el parámetro `accidentStart` y definiendo una `accidentDuration`, lo que permite observar en tiempo real cómo la detención del nodo afecta el flujo vehicular y se genera la propagación de mensajes de seguridad hacia la RSU.

Ajustes en el archivo `omnet.ini` de Veins

Ajuste el tamaño del mapa:

```
#####
#           Simulation parameters           #
#####
*.playgroundSizeX = 2500m
*.playgroundSizeY = 2500m
*.playgroundSizeZ = 50m
```

Ajuste la ubicación del RSU

```
#####
#           RSU SETTINGS                   #
#####
*.rsu[0].mobility.x = 2000
*.rsu[0].mobility.y = 2000
*.rsu[0].mobility.z = 3
```

Modifique el vehículo que genera el accidente y el momento del mismo.

```
#####
#           Mobility                       #
#####
*.node[*].veinsmobility.x = 0
*.node[*].veinsmobility.y = 0
*.node[*].veinsmobility.z = 0
*.node[*].veinsmobility.setHostSpeed = false
*.node[*0].veinsmobility.accidentCount = 1
*.node[*0].veinsmobility.accidentStart = 73s
*.node[*0].veinsmobility.accidentDuration = 50s
```

2. Introducción al Simulador de movilidad Urbana (SUMO)

*En esta práctica el estudiante instalará la plataforma de simulación de movilidad urbana (SUMO) y ejecutará una simulación sencilla. **Modalidad:** Trabajo Individual. **Recursos:** Computador personal con sistema operativo Linux, máquina virtual (VM) o Contenedor (Docker). Recomendado Ubuntu 22 <https://ubuntu.com/download/desktop>.*

2.1 Objetivos

1. Utilizar los archivos binarios para instalar el simulador sobre el sistema operativo Linux (Guía realizada sobre - Ubuntu 22.04.3 LTS).
2. Utilizar la tecnología de contenedores para instalar SUMO a partir de los archivos binarios.
3. Comprender las funciones y estructura de los archivos de simulación para correr una simulación de movilidad vehicular en SUMO.
4. Ejecutar simulaciones de tráfico tanto a través de la interfaz de línea de comandos (CLI) como mediante la interfaz gráfica de usuario (GUI).
5. Desarrollar escenarios de movilidad complejos en entornos 2D y 3D utilizando las herramientas avanzadas de edición de SUMO.

2.2 Introducción

SUMO (Simulación de Movilidad Urbana), es un paquete de simulación de tráfico multimodal (i.e., vehículos livianos, transporte público, peatones, bicicletas, etc) continuo, microscópico (i.e., configuración por nodo). SUMO es una suite de simulación de tráfico gratuita y de código abierto que incluye diferentes herramientas para la generación de tráfico, estadísticas de movilidad y visualización. SUMO incluye modelos de movilidad realista, cambio de línea, emisiones de contaminantes, entre otros. Además, incluye la interface TraCI para control en línea de la simulación.

SUMO es ampliamente utilizado en diferentes temáticas de investigación:

- Evaluación de los ciclos semafóricos.
- Optimización de rutas.
- Reducción de emisiones contaminantes.
- Conducción autónoma.
- Predicción y mejora de tráfico vehicular.

- Evaluación de redes vehiculares (VANETs).
- Manejo cooperativo (Platooning).
- Seguridad vial y análisis de riesgos.
- Planificación de rutas de transporte público.

2.3 Materiales

- **Ordenador** con sistema operativo Linux (Ubuntu 22.04.3 LTS), máquina virtual o soporte para contenedores (Docker engine).

2.4 Instrucciones

Valide con el docente

Esta práctica está estructurada en cuatro etapas. Tras concluir cada una de ellas, es indispensable solicitar la validación del docente. Dado que se requiere la entrega de un informe técnico, se recomienda documentar el proceso mediante capturas de pantalla de cada procedimiento realizado.

1. **Parte I:** Instalación de SUMO,
2. **Parte II:** Generación de archivos de simulación de SUMO,
3. **Parte III:** Ejecución de una simulación usando las consolas CLI y GUI de SUMO,
4. **Parte IV:** osmWebWizard.

2.4.1 Parte I: Instalación de SUMO

En esta primera parte vamos a instalar la suite de SUMO. Existen varias formas para instalar SUMO, incluso sobre diferentes sistemas operativos <https://sumo.dlr.de/docs/Downloads.php>; sin embargo, en esta sección presentamos un resumen de la instalación manual de la suite de SUMO sobre Ubuntu a partir de los binarios. Una guía completa se puede encontrar en https://sumo.dlr.de/docs/Installing/Linux_Build.html.

1. Actualizar los paquetes del sistema operativo.

```
#sudo apt update
#sudo apt upgrade
```

2. Instalar prerequisites de SUMO:

```
#sudo apt install git cmake python3 g++ libxerces-c-dev libfox-1.6-dev
libgdal-dev libproj-dev libgl2ps-dev python3-dev swig default-jdk
maven libeigen3-dev

#sudo apt install python3-pandas python3-rtree python3-pyproj
```

Descripción general de prerequisites:

- git: permite clonar repositorios de Github
- cmake: g++ generadores y compiladores,
- libxerces: manejo de archivos xml
- libfox-1.6-dev: manejo de GUI de SUMO
- libgdal-dev: para manejo de formatos geoespaciales,
- libproj-dev: soporte para la geoconversión y referencias
- libgl2ps-dev: usado por OpenGL,
- ccache: acelerar las construcciones,
- ffmpeg-devel: para manejo de vídeo,
- libOpenSceneGraph-devel: para la GUI en 3D,
- gtest: para pruebas de unidad, no utilice 1.13 o posterior,
- gettext: manejo de idiomas (interno),
- texttest, xvfb y tkdiff: para los ensayos de aceptación,
- copos, un estilo y autopep para la comprobación de estilo,
- swig: conector de programas de bajo a alto nivel,
- python3-dev: entorno para python3,
- jdk: entorno de java,
- maven: gestor de proyectos de software,
- libeigen3-dev: para manejo de matrices en C.

3. Una vez instalados los prerequisites, es momento de obtener los binarios de SUMO. Para esto, nos dirigimos al repositorio de Github del proyecto <https://github.com/eclipse-sumo/sumo>. Se recomienda descargar SUMO dentro del directorio `/opt/` de Linux, propio para instalar aplicaciones de terceros y opcionales.

```
#git clone --recursive https://github.com/eclipse-sumo/sumo
```

Note que este procedimiento descargará la versión más reciente (1.19.0).

4. Luego ha de instalar algunas librerías comunes de python (e.g., pandas). SUMO incluye un archivo de estos requerimientos y es necesario estar ubicados dentro de la carpeta **sumo** descargado en el paso anterior, con el fin de llamar al archivo `requirements.txt`:

```
pip install -r tools/requirements.txt
```

5. Antes de compilar el proyecto ha de definir la variable de entorno permanente `SUMO_HOME`. Este paso es muy importante ya que durante el curso se requiere que el sistema operativo conozca donde se encuentra los binarios del programa. Puede usar el comando de Linux **pwd** para ver el path completo donde se encuentra SUMO.

Es necesario modificar los archivos `/etc/bash.bashrc` y `/home/<user>/.bashrc`. Ha de agregar el path al SUMO al final de los archivos mencionados:

```
export SUMO_HOME="/home/user/sumo-version/"
```

Puede verificar que la variable de entorno se encuentre correctamente configurada con:

```
echo $SUMO_HOME
```

6. Finalmente, ha de crear una carpeta **build** dentro la carpeta raíz SUMO. Se ubica dentro de la carpeta **build** y ejecuta uno a uno los siguientes comando que construyen los binarios de SUMO:

```
cmake -B build .
cmake --build build -j $(nproc)
```

El comando `nproc` es el número de procesadores de su PC. Este proceso puede tomar varios minutos.

7. Puede verificar la instalación abriendo una terminal de Linux y ejecutando `sumo` que mostrará la versión instalada.

2.4.2 Instalación en Contenedores Docker

El despliegue del entorno de simulación (SUMO, OMNeT++ y Veins) puede realizarse mediante tres enfoques principales, cada uno con distintas implicaciones en términos de rendimiento y facilidad de configuración:

- **Contenedores (Docker):** Es la opción recomendada para entornos académicos. Consiste en utilizar una imagen preconfigurada que encapsula todas las dependencias y versiones compatibles de las herramientas. Su principal ventaja es la reproducibilidad, ya que elimina conflictos con las librerías del sistema operativo anfitrión y permite un despliegue inmediato.
- **Instalación Nativa:** Implica la compilación manual de cada componente sobre el sistema operativo (generalmente distribuciones GNU/Linux). Requiere la gestión directa de dependencias como el compilador `gcc`, librerías gráficas `Qt` y entornos de ejecución de `Python`. Aunque ofrece el máximo rendimiento de hardware, es proclive a errores de compatibilidad entre versiones.
- **Máquinas Virtuales (VM):** Consiste en el uso de un sistema operativo completo (comúnmente una imagen `.ova`) que ya incluye el software instalado. Es la alternativa más sencilla, aunque presenta una sobrecarga (*overhead*) considerable en el uso de memoria RAM y CPU en comparación con las opciones anteriores.

2.5 Instalación del framework en contenedores Docker

Para facilitar el despliegue de OMNeT++, Veins y SUMO para la simulación de redes vehiculares y heterogéneas, se han puesto a disposición dos opciones para los estudiantes:

1. **Dockerfile:** el repositorio en GitHub (<https://github.com/Pbarbecho/VANETS.git>) contiene el archivo `Dockerfile` necesario para construir la imagen del entorno. Requiere unos minutos para descargar y compilar las aplicaciones. El estudiante ha de construir el contenedor en base al `dockerfile` compartido:

```
docker build -t oppveinsumo:latest .
```

2. **Docker Image:** puede descargar directamente la imagen compartida en **DockerHub**. Considere que la imagen pesa alrededor de 5GB.


```
docker pull pbarbecho/vanets
```

En ambos casos se utiliza como base **Ubuntu 22.04** y se destacan las siguientes configuraciones para garantizar la compatibilidad entre OMNeT++ 6.3.0, SUMO 1.12 y Veins 5.3.1 (<https://veins.car2x.org/download/#compatibility>):

1. **Dependencias del Sistema y Gráficas:** Se instalan herramientas de compilación (*clang*, *make*, *build-essential*), librerías para el manejo de XML y compresión, y el stack completo de **Qt6** junto con bibliotecas de renderizado (*Mesa*, *OpenGL*, *OpenSceneGraph*). Esto es fundamental para que la interfaz gráfica de OMNeT++ (Qtenv) y SUMO-GUI funcionen correctamente mediante redirección X11. Además, se instalan directamente los binarios de **SUMO** y sus herramientas asociadas (*netconvert*, *netedit*, etc.).
2. **Ecosistema Python:** Se configuran los módulos necesarios para el análisis de resultados en OMNeT++, incluyendo *Matplotlib*, *NumPy*, *Pandas* y *SciPy*, asegurando que el entorno de análisis estadístico esté listo para su uso.
3. **Compatibilidad de Escritorio:** Se implementan *scripts* de sustitución para las herramientas *xdg-utils*. Esto evita errores comunes en entornos de contenedores donde no existe un gestor de menús de escritorio real, permitiendo que el proceso de instalación de OMNeT++ finalice sin interrupciones.
4. **Variables de Entorno:** Se definen rutas críticas como SUMO_HOME y se configuran parámetros de estabilidad gráfica como LIBGL_ALWAYS_SOFTWARE=1, lo cual fuerza el renderizado por software para evitar problemas de compatibilidad con drivers de video específicos del host.

Nota de Instalación

Para ejecutar aplicaciones gráficas desde el contenedor, asegúrese de tener instalado un servidor X (como Xming o VcXsrv en Windows, o XQuartz en macOS) y configurada correctamente la variable de entorno DISPLAY.

2.5.1 Parte II: Generación de archivos de simulación de SUMO

Una vez instalado SUMO, el primer paso es generar el escenario de simulación. Para construir el escenario, usamos las herramientas pre-instaladas en SUMO. Podemos ver una clasificación éstas herramientas en azul en la Fig. 2.1. Además, la Fig. 2.1 muestra los archivos de entrada y salida de cada herramienta, en verde.

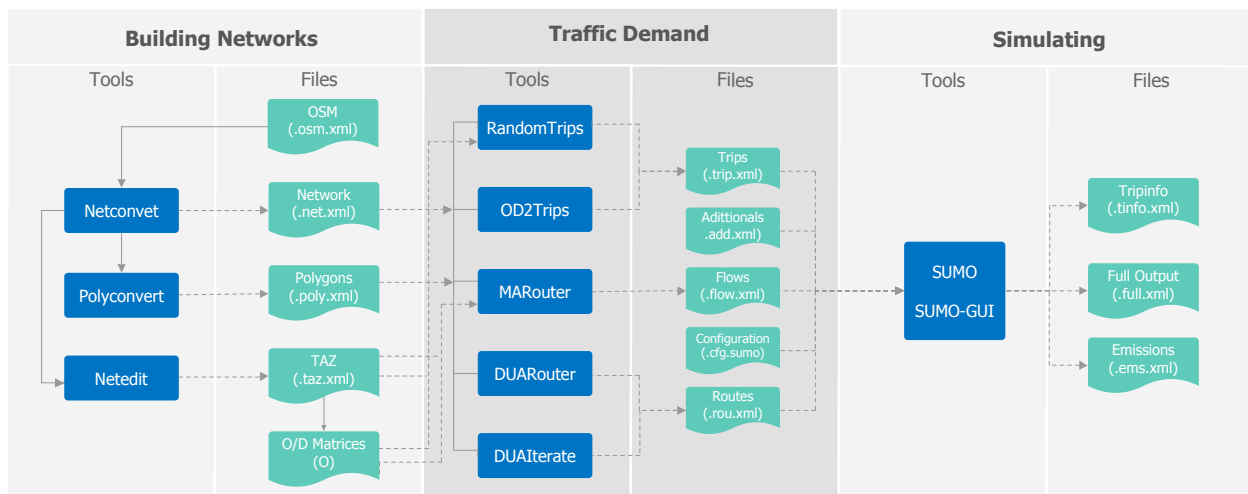


Figure 2.1: Flujo de trabajo de simulación de movilidad de tráfico. Herramientas integradas SUMO y archivos relacionados para realizar simulaciones. Tomado de [1].

Tenemos 3 grupos principales de herramientas, según sus funciones (ver Building Networks, Traffic Demand y, Simulating en la Fig. 2.1):

1. **El primer grupo de herramientas incluye:** red de carreteras, elementos de movilidad (ej., semáforos) y polígonos como edificios, parques etc. Los polígonos son de especial interés cuando evaluamos redes inalámbricas (modelos de obstrucción). En la Fig. 2.1, a la izquierda tenemos las diferentes herramientas de SUMO para generar la red de carreteras (**Netconvert**), los polígonos (**Polyconvert**). Si deseamos modificar gráficamente el mapa, lógica de semáforos, o crear zonas de interés, podemos usar un editor gráfico llamado **Nedit**.
2. **El segundo grupo de herramientas incluye:** generación de la demanda de tráfico. Aquí se definen las rutas de los vehículos, cantidad de vehículos, tipo de vehículos (transporte público, pesados, livianos, bicicletas, peatones). En la Fig. 2.1, en el centro tenemos las herramientas para generación de tráfico.
3. **El tercer grupo de herramientas incluye:** interfaces de simulación. Tenemos dos interfaces: línea de comandos (CLI), gráfica (GUI).

A continuación introducimos algunas de las herramientas que utilizaremos durante la práctica, según la clasificación de la Fig. 2.1:

• Building Networks:

- **Netconvert:** Esta herramienta convierte un archivo de mapa de entrada OpenStreetMaps (.osm) en un archivo de red de carreteras legible por SUMO. Este archivo se guarda en ASCII con un formato XML simple conocido como archivo de red (.net.xml). La herramienta netconvert proporciona un conjunto de opciones de procesamiento (por ejemplo, vías cortadas, pasos a desnivel, rotondas, etc.). Este archivo es obligatorio.
- **Polyconvert:** Esta herramienta genera todos los polígonos (por ejemplo, edificios, ter-

Tool	Inputs	Outputs	Traffic assignment method
OD2Trips (OD2)	Network, O/D Matrices, Additional	Trips	Incremental
DUARouter (DUAR)	Network, Trips, Additional	Routes	Empty-network
MARouter (MAR)	Network, O/D Matrices, Additional	Flows	Incremental
RandomTrips (RT)	Network, Additional	Trips	Incremental
DUALterate (DUAI)	Network, Trips, Additional	Routes	Iterative

Figure 2.2: Herramientas de generación de tráfico de SUMO. Tomada de [2]

renos, etc.) a partir de la fuente del mapa ingresada (.osm). El archivo de salida (.poly.xml) contiene todas las formas geométricas del mapa. Este archivo es opcional.

- *Netedit*: Esta herramienta permite a los usuarios editar/crear mapas personalizados. Viene con una interfaz gráfica de usuario donde los usuarios pueden editar las propiedades de los elementos de un mapa, como carreteras, semáforos, etc.

- **Herramientas de generación de movilidad de tráfico:**

- *RandomTrips*: Está destinado a implementaciones rápidas. Aquí, los puntos de origen/destino se seleccionan aleatoriamente en el mapa y los vehículos se distribuyen uniformemente dentro de un período de tiempo.
- *MARouter*: Generar una demanda de tráfico macroscópica. Durante el proceso de generación de tráfico se considera la distribución de la ruta, es decir, cada ruta incluye la probabilidad de ser seleccionada.
- *DUARouter*: Genera una lista de rutas que incluye la ruta completa entre los puntos de origen y destino. Utiliza el algoritmo de Dijkstra para calcular la ruta más corta. Además, el método de asignación de tráfico considera una red vacía cada vez que se genera un vehículo, lo cuál puede decantar en congestión durante la simulación (dead-lock).
- *DUALterate*: Esta herramienta utiliza un método de asignación llamado iterativo, que intenta calcular el equilibrio. Esto significa que las rutas de los vehículos se generan con el mínimo coste (por ejemplo, tiempo de viaje). Esto se hace llamando iterativamente a la herramienta *DUARouter*. Las situaciones de congestión son menos probables.
- *OD2Trips*: Genera una lista de definiciones de viaje según la cantidad de vehículos a insertar en la simulación dentro del intervalo de tiempo codificado en un archivo adicional llamado O/D matrices. Al igual que la herramienta *RandomTrips*, utiliza un método de asignación incremental.

En la Fig. 2.2, se presenta un resumen de los archivos de entrada y salida esperados en cada herramienta de generación de tráfico. En este punto, se recomienda revisar el artículo [2] que amplía los modelos y diferencias entre las diferentes herramientas de generación de tráfico como *Duarouter*, *Randomtrips*, *MARouter*, *OD2Trips*, etc.

Finalmente, un escenario de simulación en SUMO se compone por los menos de tres archivos (ver Fig. 2.3). La extensión de los archivo es .xml. Note que el archivo .poly es opcional.

Veamos una descripción de los archivos presentados en la Fig. 2.3:

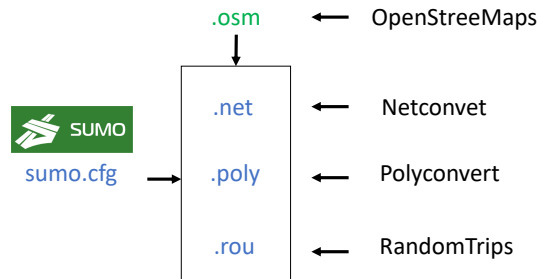


Figure 2.3: Archivos de simulación de SUMO.



Figure 2.4: Seleccionar área y exportar mapa en formato .osm.

- **Archivo de red (.net):** Este archivo es generado con la herramienta *Netconvert* y contiene toda la información acerca del mapa, carreteras y calles.
- **Archivo de polígonos (.poly):** Este archivo es generado con la herramienta *Polyconvert* contiene toda la información acerca de edificios y otros obstáculos como parques que existen en el mapa.
- **Archivo de viajes (.rou):** Este archivo contiene toda la información referente a las rutas de vehículos dentro de la simulación. La manera más sencilla para generar este archivo es usando la herramienta *RandomTrips*.
- **Archivo de Configuración (.cfg):** Los archivos antes mencionados están organizados en un archivo de configuración único donde se indica el tiempo de simulación, los directorios de ubicación de los archivos .net y .rou entre otros.

A continuación se describen los pasos para generar los archivos necesarios (ver Fig. 2.3) para correr una simulación:

1. Antes de comenzar a generar los archivos, se sugiere crear una carpeta por proyecto, donde se almacenarán todos los archivos de simulación.
2. El primer archivo a generar es el mapa (ver .osm en Fig.2.3). SUMO soporta diferentes formatos, vamos a exportar un mapa descargado de OpenStreetMaps (OSM), <https://www.openstreetmap.org/>. Seleccione una sección pequeña de su ciudad y haga clic en "Exportar" y luego seleccione la opción "Seleccionar manualmente un área diferente", ver Fig. 2.4.

Nota: A través de este método (OSM), solo se permite exportar un número máximo de 50000

nodos, incluyendo carreteras, señales de tráfico, rutas de transporte público, etc.

3. Vamos a generar el siguiente archivo que corresponde a la red de carreteras (.net) (ver Fig. 2.3). Para esto, usamos la herramienta *netconvert*:

```
netconvert --ramps.guess --remove-edges.isolated --edges.join --geometry.remove --osm-files map.osm -o bcn.net.xml
```

Donde:

- `-ramps.guess` permite predecir rampas que no se lograron especificar en el mapa.
- `-remove-edges.isolated` elimina los bordes aislados del mapa.
- `-edges.join` fusiona nodos que están cercanos entre sí.
- `-geometry.remove` reemplaza los nodos que solo definen la geometría del borde por puntos de geometría.
- `-osm-files` indica la ruta y el nombre al archivo .osm (descargado desde OSM).
- `-o` indica la ruta y el nombre del archivo de salida.

La herramienta tiene varias opciones que se pueden agregar, por ejemplo, excluir rutas de tranvía. Para conocer las opciones nos referimos a la documentación [3]. La entrada de *netconvert* es el archivo osm previamente descargado y la salida es el archivo de la red (.net), por ejemplo *bcn.net.xml*, donde *bcn* es un nombre cualquiera que le damos al mapa y se mantiene las extensiones .net.xml.

4. Ahora generamos el archivo de los polígonos (ver Fig. 2.3). Invocamos la herramienta *polyconvert*:

```
polyconvert --net-file bcn.net.xml --osm-files map.osm -o bcn.poly.xml
```

Donde:

- `-net-file` indica la ruta y el nombre del archivo de la red (net).
- `-osm-files` indica la ruta y el nombre al archivo .osm (descargado desde OSM).
- `-o` indica la ruta del archivo de polígonos (edificios, parques, etc.) de salida con extensión .poly.xml.

Otras opciones de *polyconvert* se encuentran en [4].

5. Ahora generamos las rutas (.rou.xml). Usamos la herramienta *RandomTrips* que hace referencia a un script en Python. ***RandomTrips*** está ubicado en el directorio: */opt/sumo/tools/*. Corremos el script de Python *randomTrips.py*.

```
python3 randomTrips.py -n bcn.net.xml -r bcn.rou.xml
```

Donde:

- `-r` genera un archivo de rutas con DUAROUTER. Archivo de rutas de salida con extensión .rou.xml.
- `-n` indica la ruta y el nombre del archivo de la red (.net).

Una opción de utilidad es `-p`. Esta opción permite generar tráfico en el que consten *n* vehículos distribuidos uniformemente en un período de tiempo. `-p` se obtiene de:

$$p = \frac{t_1 - t_0}{n} \quad (2.1)$$

Donde, t_1 y t_0 son el tiempo final e inicial de la simulación, respectivamente. Por ejemplo, 3600 segundos, comenzando en el segundo 0. Reemplazando en la ecuación 2.1 se obtiene el valor de $p=36$.

$$p = \frac{3600}{100} = 36 \quad (2.2)$$

Finalmente, el comando sería el siguiente:

```
python3 randomTrips.py -b 0 -e 3600 -p 36 -n bcn.net.xml -r bcn.rou.xml
```

Otras opciones de la herramienta *RandomTrips* están disponibles en [5].

6. Finalmente, necesitamos el archivo de configuración de sumo (ver Fig. 2.3). Este archivo contiene a todos los archivos necesarios para la simulación (.net, .poly y .rou).

El archivo sumo.cfg (ver Código 2.1) tiene un formato .xml e incluye directivas del escenario de simulación. La configuración de este archivo se divide en bloques. En el Código 2.1 identificamos el bloque *<input>*, *<processing>*, y *<time>*.

La sección de entrada *<input>* establece la ubicación de los archivos de la red, rutas de vehículos y polígonos. La sección de procesamiento *<processing>* incluye la directiva *<ignore-route-errors>*. Esto permite ignorar los errores que pueden generarse por rutas que no se pueden completar (no hay conexión entre el punto origen y destino). Finalmente, la sección de tiempo establece el tiempo de inicio, finalización y la duración de los pasos de tiempo de la simulación.

```
<?xml version="1.0" encoding="UTF-8"?>
  <input>
    <net-file value="bcn.net.xml"/>
    <route-files value="bcn.rou.xml"/>
    <additional-files value="bcn.poly.xml"/>
  </input>

  <processing>
    <ignore-route-errors value="true"/>
  </processing>

  <time>
    <begin value="0"/>
    <end value="10000"/>
    <step-length value="1"/>
  </time>
```

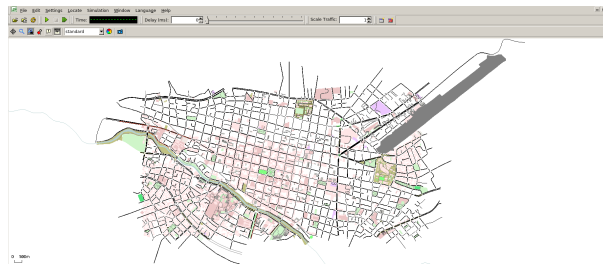


Figure 2.6: Archivo de configuración cargado.

```
</configuration >
```

Listing 2.1: Archivo `.sumo.cfg`.

2.5.2 Parte III: Generación de una simulación usando las consolas CLI y GUI de SUMO.

A continuación, se describen los pasos necesarios para correr una simulación usando la CLI y GUI de SUMO.

1. **Simulación usando la CLI:** necesitamos indicar el archivo de configuración con extensión `sumo.cfg`.

```
#sumo -c bcn.sumo.cfg
```

Donde, `-c` indica la ubicación y el nombre del archivo de configuración `xx.sumo.cfg`.

```
sumo@sumo-VirtualBox:~/sumo/bin$ ./sumo -c cuenca.sumo.cfg
Step #0.00 (12ms -> 83.33*RT, -83.33UPS, vehicles TOT 1 ACT 1 BUF 0)
Warning: No connection between edge '118366336#2' and edge '45429539#0' found.
Warning: No route for vehicle '8' found.
Warning: No connection between edge '-48202935' and edge '334948974#6' found.
Warning: No route for vehicle '23' found.
Warning: No connection between edge '399760675#0' and edge '256907298#0' found.
Warning: No route for vehicle '42' found.
Warning: No connection between edge '400449634#1' and edge '334948974#7' found.
Warning: No route for vehicle '95' found.
Step #100.00 (2ms -> 500.00*RT, -49500.00UPS, vehicles TOT 100 ACT 99 BUF 1)
Warning: No connection between edge '535309213#0' and edge '48859314#3' found.
Warning: No route for vehicle '132' found.
Warning: No connection between edge '400463706#1' and edge '334948974#2' found.
Warning: No route for vehicle '144' found.
Warning: No connection between edge '640993657#1' and edge '311212627#3' found.
Warning: No route for vehicle '178' found.
Warning: No connection between edge '45429545#2' and edge '42201281#1' found.
Warning: No route for vehicle '181' found.
Warning: No connection between edge '48537834' and edge '400713354' found.
Warning: No route for vehicle '199' found.
Step #200.00 (3ms -> 333.33*RT, -62333.33UPS, vehicles TOT 200 ACT 187 BUF 1)
Warning: No connection between edge '49577750' and edge '-972644144#4' found.
```

Figure 2.5: Simulación en marcha desde la línea de comandos de Linux.

Como vemos en la Fig. 2.5, se presentan ciertas advertencias refiriéndose a rutas sin conexión. La CLI suele utilizarse para ejecutar simulaciones largas y pesadas.

2. **Simulación usando la GUI:** Para ejecutar la simulación con interfaz gráfica (GUI), abrimos una terminal de Linux y ejecutamos `sumo-gui`.
3. En el menú superior de la GUI de SUMO (ver Fig. 2.6), seleccionamos *File* → *Open Simulation* y buscamos el archivo de configuración del escenario `xx.sumo.cfg`. En la parte inferior de la GUI se muestran posibles errores al intentar cargar los archivos `.net` y `.poly` definidos en el archivo de configuración `.cfg`.
4. Finalmente, para ejecutar la simulación consideramos la velocidad de simulación en la parte superior (recuadro rojo de la Fig. 2.7). Luego, es necesario dar clic en la opción *Run* (recuadro azul de la Fig. 2.7) y la simulación estará en marcha (ver Fig. 2.8).

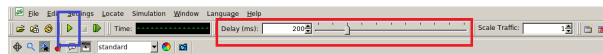


Figure 2.7: Controles de la velocidad de simulación.

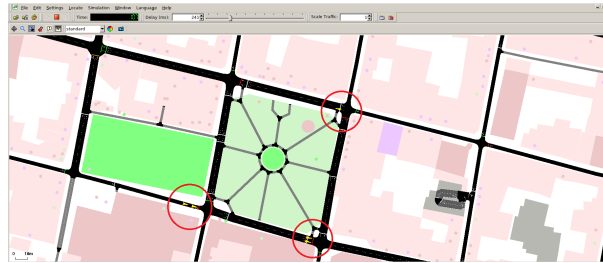


Figure 2.8: Simulación en marcha desde la interfaz gráfica de SUMO.

2.5.3 Parte IV: osmWebWizard

Dentro de la suite de herramientas de SUMO, una herramienta particularmente útil es osmWebWizard. Este componente permite a los usuarios aprovechar los datos geospaciales disponibles en OpenStreetMap (OSM) para generar escenarios de simulación realistas y detallados y sin limitación del número de nodos.

OSMWebWizard simplifica significativamente el proceso de generación de archivos de simulación de SUMO. Se basa en una interfaz gráfica Web (se recomienda usar Firefox) que guía a los usuarios a través de los pasos necesarios para importar mapas, definir atributos de la red vial, y configurar parámetros de simulación.

En esta parte de la práctica, exploramos cómo utilizar el OSMWebWizard para crear escenarios de simulación en SUMO.

1. La herramienta se localiza dentro del directorio *tools* de SUMO en la ruta:
/opt/sumo/tools. OSMWebWizard es un script en Python `osmWebWizard.py` y para ejecutarlo escribimos en la terminal:

```
# ./osmWebWizard.py
```

Se abrirá un navegador web con el mapa de OSM (ver Fig. 2.9). Se puede modificar la ubicación (e.j., Quito). Podemos seleccionar la opción *Seleccionar Área* o simplemente el mapa en pantalla será el mapa que se exporte. De igual manera, podemos marcar la opción *Add Polygons* si necesitamos generar el archivo de polígonos (opcional). En el icono del carro, nos aparece un segundo menú que nos permite configurar el número de vehículos en la simulación, así como otro tipo de vehículos (ver Fig. 2.10).

2. Una vez presionemos en Generate Scenario (botón verde en Fig. 2.9), podemos mirar en consola los comandos que se ejecutan automáticamente. Finalizado el proceso, se nos presenta un resumen en consola y se agrega una carpeta dentro del directorio /opt/sumo/tools/ cuyo nombre corresponde a la fecha en la que se generaron los archivos. Además, se ejecutará automáticamente la simulación en la GUI de SUMO.

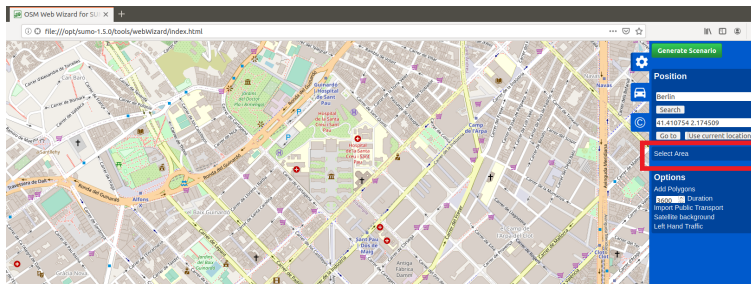


Figure 2.9: osmWebWizard tool.

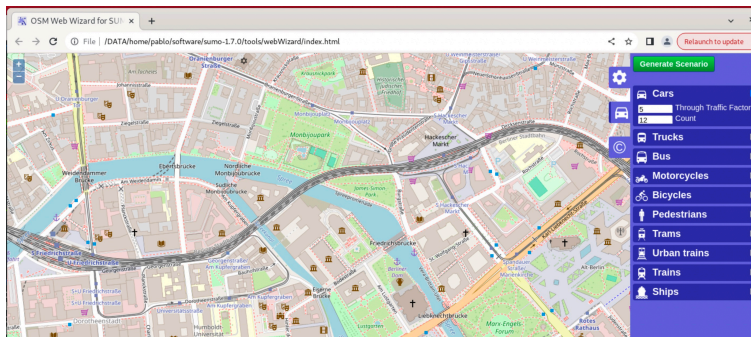


Figure 2.10: osmWebWizard tool.

En este punto es importante identificar los archivos generados dentro de la carpeta. En la Fig. 2.11 se listan los archivos generados de manera automática.

A continuación se presenta una descripción de cada archivo:

- `build.bat` es un bash script que contiene el comando para correr la herramienta `randomTrips.py` indicando cada una de las opciones agregadas.
- `osm.netccfg` es el archivo de configuración usado para el `netconvert`. Construye el archivo final `.net`.
- `osm.net.xml` es el archivo de carreteras final `.net`.
- `osm.poly.xml` es el archivo de configuración final de polígonos `.poly`.
- `osm_bbox.osm.xml` es el archivo del mapa en formato `.osm`.
- `osm.passenger.trips.xml` contiene las rutas de los vehículos (pasajeros).
- `osm.polycfg` contiene la información del archivo usado para generar los polígonos.
- `osm.sumocfg` es el archivo de configuración de la simulación de `sumo`.

```
-rw-rw-r-- 1 pablo pablo 354 Mar 5 03:35 build.bat
-rw-rw-r-- 1 pablo pablo 7354804 Mar 5 03:34 osm.net.xml
-rw-rw-r-- 1 pablo pablo 1212 Mar 5 03:34 osm.netccfg
-rw-rw-r-- 1 pablo pablo 59092 Mar 5 03:35 osm.passenger.trips.xml
-rw-rw-r-- 1 pablo pablo 1416755 Mar 5 03:34 osm.poly.xml
-rw-rw-r-- 1 pablo pablo 691 Mar 5 03:34 osm.polycfg
-rw-rw-r-- 1 pablo pablo 851 Mar 5 03:35 osm.sumocfg
-rw-rw-r-- 1 pablo pablo 88 Mar 5 03:35 osm.view.xml
-rw-rw-r-- 1 pablo pablo 13234119 Mar 5 03:34 osm_bbox.osm.xml
-rwxr-xr-x 1 pablo pablo 23 Mar 5 03:35 run.bat
```

Figure 2.11: Archivos SUMO generados.

- `osm.view.xml` contiene la configuración de la ejecución en el GUI de SUMO.
- `run.bat` es un script que contiene el comando para correr la simulación de sumo desde la interfaz gráfica.

Los archivos marcados en azul son los que nos interesa para reproducir la simulación. Podemos copiarlos a otro directorio y modificarlos o usarlos para ejecutar una nueva simulación. Recordando de la práctica anterior, en el archivo `sumo.cfg`, teníamos como entrada `<input>` los archivos: `.net`, `.rou` y `.poly`. En este caso tenemos los archivos `.net`, `trips` y `.poly`. Se observa que ahora se utiliza el archivo `.trips` es cual reemplaza al archivo `.rou`, más adelante explicaremos la diferencia entre `.trips` y `.rou`.

3. Genere nuevamente la simulación usando los archivos generados con `osmWebWizard`.

NOTA: Si al correr el script `osmWebWizard.py` salta el siguiente error:

```
osmWebWizard.py: error: typemap file "\${SUMO\_HOME}/data/typemap/  
osmPolyconvert.typ.xml" not found
```

Es necesario verificar que la variable de entorno se encuentre configurada correctamente `echo $SUMO_HOME`, y verificar la existencia del archivo `osmPolyconvert.typ.xml` en el path especificado en el error.

Objetivos

Introducción

Modelos de asignación de tráfico

Herramientas de generación de demanda vehicular

Desarrollo

Parte I: Flujos Aleatorios con RandomTrips.py

Parte II: Demanda basada en el comportamiento (TAZ y matrices OD)

Parte III: Enrutamiento dinámico duarouter

Parte IV: Análisis de modelos de generación de demanda vehicular

3. Demanda de movilidad vehicular

En esta práctica el estudiante define las trazas de movilidad basadas en modelos de asignación de tráfico. Se utilizan diferentes herramientas de demanda de tráfico incluidas en el simulador SUMO: RandomTrips, DUARouter, OD2trips, matrices OD y zonas de asignación de tráfico (TAZ).

3.1 Objetivos

- Diferenciar entre flujos y rutas de vehículos.
- Comprender los modelos de generación en los que se basan las herramientas de SUMO.
- Generar tráfico multimodal (vtypes)

3.2 Introducción

Como vimos en la práctica anterior, para ejecutar una simulación vehicular necesitamos como mínimo la red de carreteras `.net.xml`, las rutas de los vehículos `.rou.xml`, y el archivo de configuración de la simulación `.sumocfg`. En esta práctica vamos a experimentar diferentes herramientas de generación de tráfico que influyen directamente en la distribución de vehículos en mapa. Además vamos generar un tráfico multimodal que incluya diferentes clases de vehículos. Para esto agregamos un archivo de definición de las características de cada tipo de vehículo `vtypes` (ver Fig. 3.1).

3.3 Modelos de asignación de tráfico

A continuación, se describen los principales modelos de asignación de tráfico:

- **Asignación de Red Vacía (Empty-network):** Las rutas de los vehículos se calculan considerando que la red está totalmente libre de otros vehículos. No tiene en cuenta la densidad del tráfico en las carreteras. Crea rutas basado en el algoritmos de Dijkstra. En escenarios densos, es probable que genere congestión vehicular dado que siempre prefiere la ruta mas corta.
- **Asignación Incremental:** El tráfico se introduce en la red en pasos sucesivos. A medida que se añade carga a la red, los tiempos de viaje se actualizan, influyendo en la elección de ruta de los vehículos que entran posteriormente.

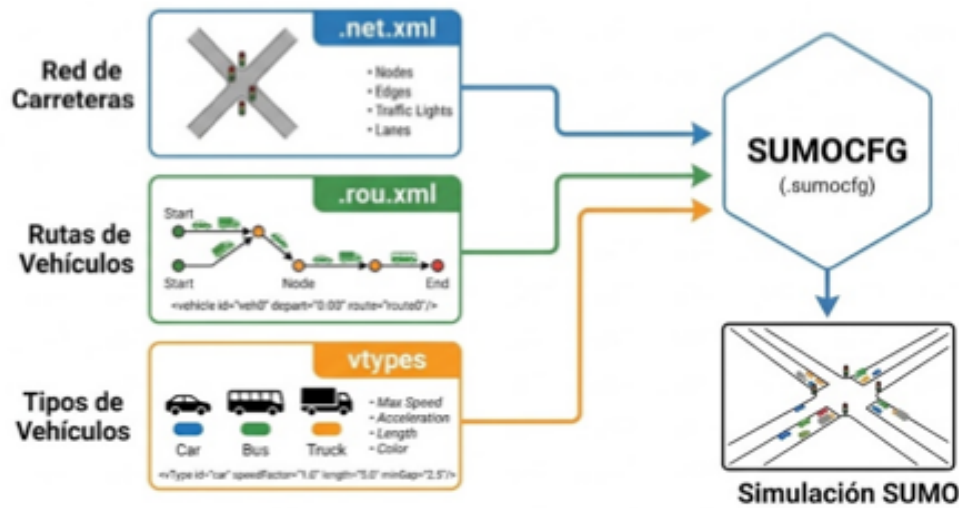


Figure 3.1: Archivos de entrada de simulación definidos en el input del `simulation.sumocfg`

- **Asignación Iterativa:** Se basa en un ciclo de retroalimentación donde la simulación se ejecuta repetidamente. En cada iteración, una parte de los vehículos cambia sus rutas basándose en los costos (tiempos) medidos en la iteración anterior. Converge hacia un **equilibrio**, donde los tiempos de viaje se optimizan globalmente.

3.4 Herramientas de generación de demanda vehicular

A continuación se presenta un resumen de las herramientas principales para la generación de movilidad en el simulador SUMO basadas en los diferentes modelos de asignación de tráfico (ver Tabla 3.1).

- **OD2Trips (OD2):** Utiliza el modelo de asignación **incremental** para generar viajes individuales a partir de matrices de origen y destino.
- **DUARouter (DUAR):** Aplica el modelo de **red vacía (Empty-network)**, donde las rutas se calculan basándose únicamente en las distancias o tiempos de viaje libres, sin considerar la carga actual de vehículos.
- **MARouter (MAR):** Implementa una asignación **incremental** orientada a la generación de flujos de tráfico a partir de datos macroscópicos (matrices O/D).
- **RandomTrips (RT):** Emplea un esquema **incremental** para crear viajes con puntos de inicio y fin aleatorios dentro de la red definida.
- **DUAIterate (DUAi):** Se distingue por su enfoque **iterativo**, ejecutando múltiples ciclos de simulación para alcanzar un equilibrio.

3.5 Desarrollo

Generar la demanda vehicular utilizando diferentes modelos de asignación de tráfico. Utilice las herramientas del simulador SUMO: RT, OD2 y DUA (ver Tabla 3.1). Analice la distribución ve-

Table 3.1: SUMO traffic generation tools.

Tool	Inputs	Outputs	Traffic assignment model
OD2Trips (OD2)	Network, O/D Matrices, Additional	Trips	Incremental
DUARouter (DUAR)	Network, Trips, Additional	Routes	Empty-network
MARouter (MAR)	Network, O/D Matrices, Additional	Flows	Incremental
RandomTrips (RT)	Network, Additional	Trips	Incremental
DUAIterate (DUAI)	Network, Trips, Additional	Routes	Iterative

hicular resultante en cada caso. Utilice las herramientas gráficas del simulador para verificar que el comportamiento del tráfico sea realista y válido. Esta validación es crítica para más adelante garantizar la fiabilidad de las pruebas de rendimiento en arquitecturas de comunicación V2V/V2I sobre la red modelada.

Valide con el docente

El desarrollo de la práctica se divide en cuatro etapas secuenciales. Una vez completadas, es obligatoria la validación de los resultados con el docente. **En el informe deberá reportar únicamente el desarrollo de la Parte IV.**

1. **Parte I:** Demanda aleatoria vehicular (`randomTrips.py`).
2. **Parte II:** Demanda basada en el comportamiento (TAZ y matrices OD).
3. **Parte III:** Enrutamiento dinámico (`duaroute`)
4. **Parte IV:** Análisis de modelos de generación de demanda vehicular

3.5.1 Parte I: Flujos Aleatorios con `RandomTrips.py`

El experimento consiste en la generación de un flujo vehicular utilizando la herramienta `randomTrips.py`, para un intervalo de una hora, el cual será sometido a un análisis de sensibilidad mediante incrementos progresivos en el volumen de vehículos. El objetivo es identificar el punto de saturación de la infraestructura vial, definido como el umbral en el que se originan bloqueos persistentes (traffic jams). El estado de saturación se diagnosticará mediante el monitoreo de eventos de teleporting, los cuales indican que la red ha excedido su capacidad operativa y los vehículos no pueden avanzar (https://sumo.dlr.de/docs/Simulation/Why_Vehicles_are_teleporting.html). Puede apoyarse de una intuición gráfica (Sumo-GUI) para validar el bloqueo de la red.

3.5.2 Parte II: Demanda basada en el comportamiento (TAZ y matrices OD)

La generación de demanda basada en patrones de comportamiento requiere la delimitación de distritos de tráfico (TAZ) sobre el área de estudio y la caracterización de los flujos de viaje entre estos. Como se ilustra en la Fig. 3.2, el flujo de trabajo se estructura en tres fases consecutivas: (i) la configuración geométrica y lógica de los archivos de distritos TAZ, (ii) la definición de matrices de Origen-Destino (O/D) que cuantifican la movilidad entre zonas, y (iii) la generación de la demanda vehicular mediante la conversión de flujos macroscópicos en viajes individuales.

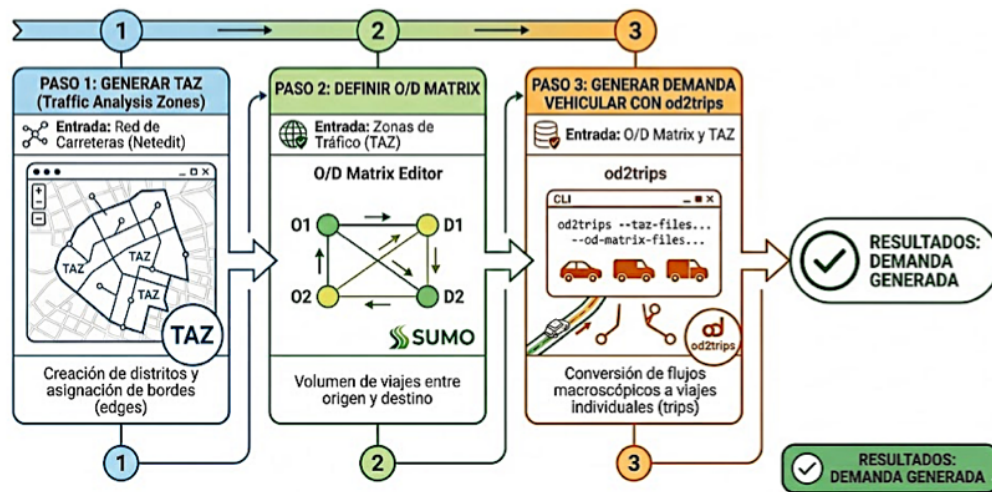


Figure 3.2: Flujo de trabajo para generar demanda vehicular basada en el comportamiento de una ciudad.

Paso 1: Distritos o zonas de asignación de tráfico (TAZ)

El método basado en distritos de tráfico (TAZ o Traffic assignment zones) https://sumo.dlr.de/docs/Demand/Importing_O/D_Matrices.html permite modelar la demanda vehicular a partir del comportamiento real de movilidad urbana. Mediante la importación de matrices de Origen-Destino (O/D), es posible cuantificar con precisión el flujo de vehículos que se desplazan desde una zona de origen *A* hacia una zona de destino *B*. Esta aproximación garantiza que la distribución del tráfico en el simulador sea representativa de los patrones de viaje observados en la ciudad bajo estudio.

Los distritos de tráfico (TAZ) se integran en la simulación como elementos de tipo `additional`. El archivo TAZ (`taz.xml`) define una entidad geográfica compuesta por un identificador único (`id`), una geometría delimitada por un polígono (`shape`) y el conjunto de bordes o carreteras (`edges`) que conforman dicha zona. Esta estructura permite al simulador asociar los flujos de la matriz O/D con la infraestructura vial correspondiente.

```
<taz id="norte" shape="299.24,943_497.24,943_497.24,1141.9_299.24,1141.9_
299.24,943" color="0,96,255" edges="-24212409"/>
<taz id="sur" shape="299.24,1143.9_497.24,1143.9_497.24,1341.9_
299.24,1341.9_299.24,1143.9" color="0,190,255" edges="24212410"/>
<taz id="este" shape="299.24,1343.9_497.24,1343.9_497.24,1541.9_
299.24,1541.9_299.24,1343.9" color="255,0,20" edges="432301136#0"/>
<taz id="oeste" shape="299.24,1543.9_497.24,1543.9_497.24,1741.9_
299.24,1741.9_299.24,1543.9" color="255,0,41" edges="-24214862#0_
24214861#3_24214862#0"/>
```

Un TAZ se puede generar de forma gráfica con la herramienta `netedit` o por medio de un script de SUMO que crea grillas de un tamaño definido (`gridDistricts.py`). La elección entre ambos métodos depende del nivel de granularidad y realismo requerido para el análisis de las comunicaciones V2V/V2I.

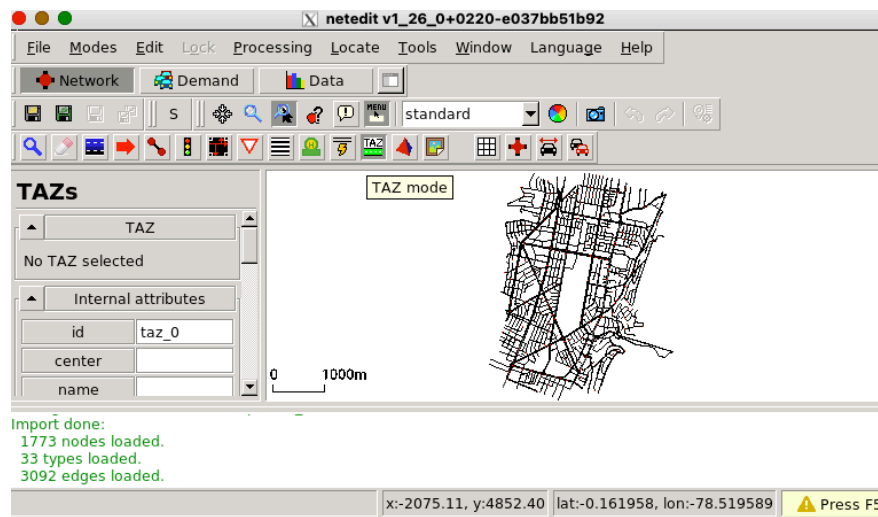


Figure 3.3: TAZ con Netedit

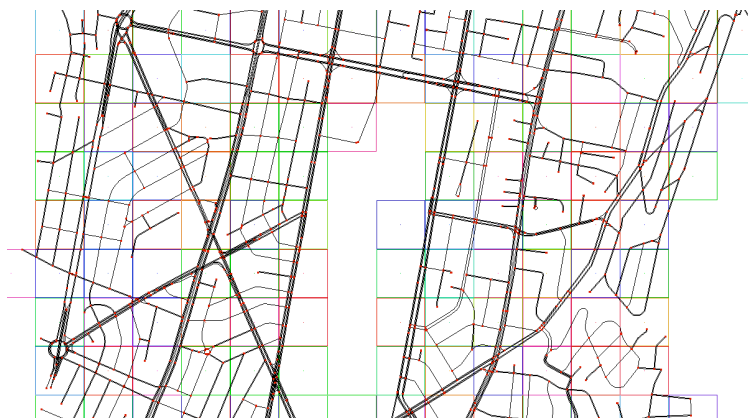


Figure 3.4: TAZ en grillas

- **TAZ con Netedit:** Es una herramienta gráfica que permite modificar el mapa de manera gráfica (ver Fig. 3.3). Basta con ejecutar `netedit` en la terminal y buscar en la interfaz TAZ mode.
- **TAZ en grillas:** Genera distritos en forma de grilla (ver Fig. 3.4). Usamos el script `gridDistricts.py` ubicado dentro de `tools/district/`. Su uso es mediante la terminal:

```
python tools/district/gridDistricts.py -n <net-file> -o <output-file> -w
300
```

Donde `w` define el tamaño en metros de la grilla. Una vez generada la grilla, se puede usar `netedit` para verificar la grilla sobre el mapa. El archivo `taz.xml` se carga como archivo del tipo `additional`.

Paso 2: Matrices de comportamiento Origen-Destino (OD)

El O-Format (u O-Matrix) es un estándar de archivos de texto plano, originario del software VI-SUM, que SUMO adopta para la descripción macroscópica de la demanda. A diferencia de los flujos aleatorios, el O-Format permite estructurar la movilidad urbana mediante una matriz que cuantifica los viajes entre distritos de tráfico (TAZ) en intervalos temporales específicos. https://sumo.dlr.de/docs/Demand/Importing_OD_Matrices.html

Para el ejemplo del TAZ con 4 distritos norte, sur, este, oeste; el archivo de demanda quedaría de la siguiente manera:

```
$OR;D2
* From-Time   To-Time
7.00 8.00
* Factor
1.00
* comments
      norte      sur      10.00
      norte      este     20.00
      sur        este      5.00
      oeste      este     14.00
```

Note que este archivo define la demanda de tráfico entre distritos y para el período entre las 7 a 8 horas. Pueden generarse varios archivos para diferentes instantes de tiempo.

Paso 3: Generación demanda basada en el comportamiento od2trips

Una vez generados los distritos TAZ y la matriz de demanda entre distritos, procedemos a generar los viajes de los vehículos. Para esto usamos la herramienta de SUMO od2trips. Veamos un ejemplo de uso de la herramienta:

```
od2trips --taz-files distritos.add.xml --od-matrix-files matriz.txt -o viajes.
trips.xml
```

Listing 3.1: Comando para convertir matriz O/D en viajes individualizados.

Como resultado obtenemos el archivo viajes.trips.xml, que contiene la cantidad de viajes entre distritos definidos en el archivo matriz.txt.

```
<trip id="9" depart="25218.39" from="-24212409" to="432301136#0" fromTaz="
norte" toTaz="este" departLane="free" departSpeed="max"/>
<trip id="5" depart="25255.65" from="-24212409" to="432301136#0" fromTaz="
norte" toTaz="este" departLane="free" departSpeed="max"/>
<trip id="19" depart="25363.02" from="-24212409" to="432301136#0" fromTaz="
norte" toTaz="este"
```

Nota: considere que puede o no existir conexión lógica entre un origen y un destino. Para esto es importante agregar la opción `<ignore-route-errors value="true"/>` en la configuración del `simulation.sumocfg`. Además, debe indicar la ubicación del archivo de los distritos TAZ como `additional-files`:

```
<input>
<net-file value="mapa.net.xml"/>
```

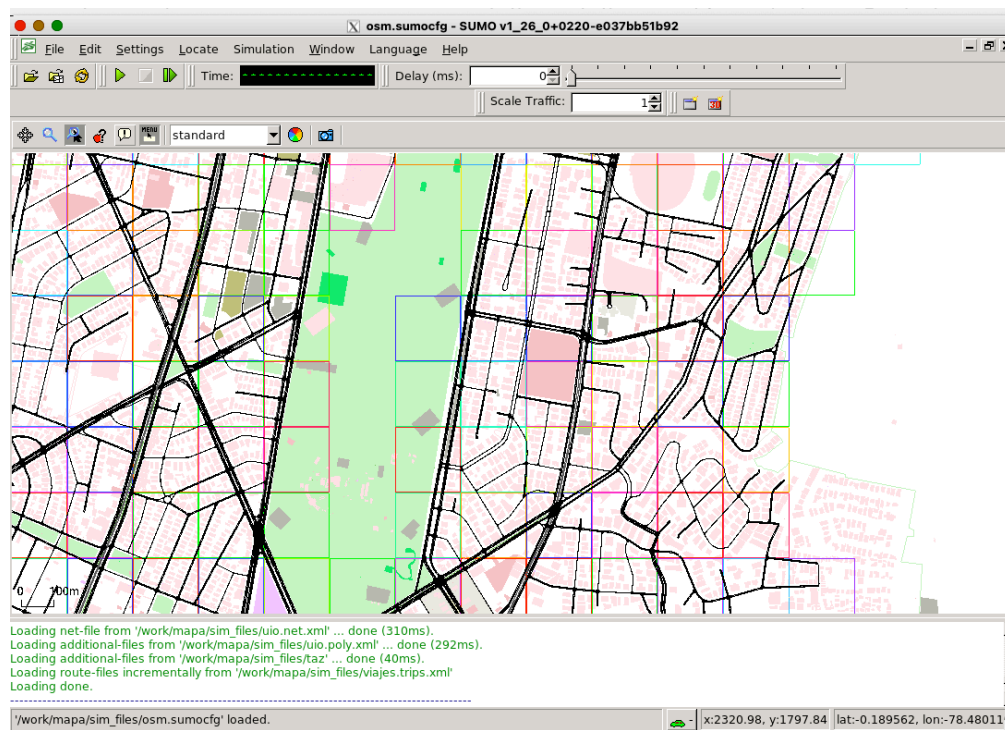



Figure 3.5: Simulación de viajes tipo trips basado en TAZ y matrices O/D

```

    <routes value="viajes.trips.xml"/>
    <additional-files value="mapa.poly.xml, taz"/>
</input>

<processing>
    <ignore-route-errors value="true"/>
</processing>

<time>
    <begin value="0" />
    <step-length value="1" />
    <end value="3600" />
</time>

```

Al ejecutar la simulación mediante la interfaz gráfica (SUMO-GUI), se obtendrá un resultado similar al ilustrado en la Fig. 3.5. En la pantalla se puede verificar si los archivos configurados en `simulation.sumocfg` se han cargado correctamente.

3.5.3 Parte III: Enrutamiento dinámico duarouter

Se utiliza para calcular las rutas más cortas y aplicar el equilibrio de usuario de manera iterativa (DUA - Dynamic User Assignment) <https://sumo.dlr.de/docs/duarouter.html>. Su objetivo teórico es alcanzar un equilibrio donde cada conductor elige la ruta que minimiza su tiempo de viaje. Al final, ningún conductor puede mejorar su tiempo cambiando de ruta unilateralmente

(Equilibrio de Nash aplicado al tráfico).

A diferencia del método basado en distritos, la generación de rutas mediante `duarouter` requiere únicamente el archivo de infraestructura vial `mapa.net.xml`. No es necesaria la inclusión de archivos adicionales como TAZ o matrices O/D, pero requiere del archivo/s de viajes `trips.xml`. El archivo resultante, `dua.rou.xml`, se define bajo el elemento de tipo `routes`. Es fundamental distinguir que, mientras las herramientas como `od2trips` generan `trips` (los cuales solo especifican origen, destino y tiempo de partida), el resultado de `DUARouter` proporciona la secuencia exacta de bordes (`edges`) que conforman la trayectoria del vehículo, garantizando una ruta válida y continua sobre la red.

Veamos un ejemplo de uso la herramienta `duarouter`:

```
duarouter --net-file mapa.net.xml --route-files viajes.trips.xml --output-file
dua.rou.xml
```

El archivo de salida `dua.rou.xml` define rutas completas y validas (existe conexión entre el origen y el destino) para cada vehículo en la simulación.

```
<routes xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="http://sumo.dlr.de/xsd/routes_file.xsd">
  <vehicle id="63" depart="25200.79" departLane="free" departSpeed="max"
    fromTaz="norte" toTaz="este">
    <route edges="-24394130#1_420636432#1_420636432#2_24394132#1_
      24394132#2_427673346#0_290622851_60265834#1_431921290_24394138_
      533107317#0_533107317#1"/>
  </vehicle>
  <vehicle id="283" depart="25202.83" departLane="free" departSpeed="max"
    fromTaz="norte" toTaz="este">
    <route edges="24394131#1_145878644#9_24394130#2_24394130#3_24394130#4_
      24394130#5_24394130#6_24215834#1_24215834#2_538986215#0_538986215#1
      _24393163#0"/>
  </vehicle>
```

3.5.4 Parte IV: Análisis de modelos de generación de demanda vehicular

Documentar en el informe: Compare la distribución espacial de los vehículos sobre el mapa para cada herramienta estudiada en esta práctica: RT, DUA y OD2. Utilice el script de visualización de SUMO `plot_net_dump.py` (ver ejemplo de la Fig. 3.6). Para cada demanda de tráfico (flujos o rutas), debe considerar al menos tres tipos diferentes de clases vehículos `vClass` (ej. Sedán-passenger, Bus, y Motocicleta) generados basados en una distribución predefinida. Por ejemplo 20% del total de los vehículos son buses, 70% son del tipo passenger, y el 10% restante son motocicletas. Finalmente, sustente sus conclusiones con una tabla donde reporte las ventajas y limitaciones de cada herramienta de generación de tráfico utilizada en la práctica.

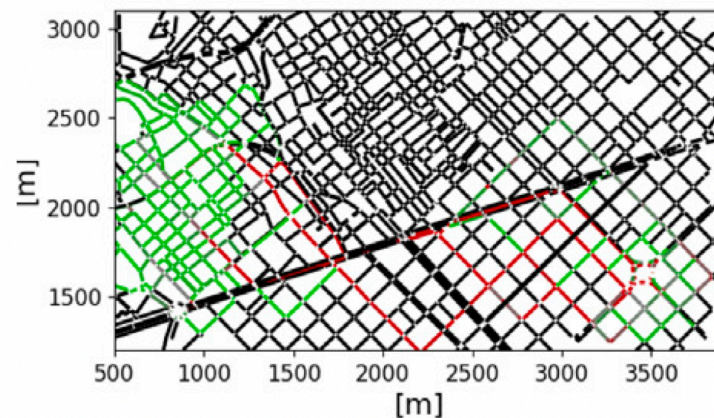


Figure 3.6: Ejemplo de la distribución de vehiculos en el mapa (edgeslane traffic)

Plots de distribución espacial de los vehículos en el mapa

Como se ilustra en la Fig. 3.6, la distribución espacial de los vehículos sobre la red vial permite identificar los niveles de carga en la infraestructura. Este análisis se fundamenta en la generación previa de estadísticas de ocupación por borde (*edges*), siguiendo los protocolos de salida de SUMO para métricas de tráfico. Dicho archivos de salida sirven como elemento de entrada para el script de post-procesamiento `plot_net_dump.py` https://sumo.dlr.de/docs/Definition_of_Vehicles%2C_Vehicle_Types%2C_and_Routes.html#abstract_vehicle_class, el cual mapea los indicadores de desempeño sobre la geometría de la red mediante una representación cromática de la densidad.

`plot_net_dump.py` se ubica en `tools/visualization/`. Este script muestra una red, donde los colores y el ancho de los bordes de la red se establecen en dependencia de los atributos de borde definidos. Los pesos de los bordes se leen a partir del archivo *edgeslane traffic*. Veamos un ejemplo de uso:

```
python3 /home/sumo/sumo/tools/visualization/plot_net_dump.py -v -n osm.net.xml
--measures density,density --xlabel [m] --ylabel [m] --default-width 1 -i
edgedata.xml,edgedata.xml --xlim 000,2600 --ylim 000,1500 --default-width
.5 --min-color-value 0 --max-color-value 5 --max-width-value 5 --min-width-
value 0 --max-width 3 --min-width .5 --colormap winter -o density-color-
map.png
```

Simulación multimodal: tipos de Vehículos (vtypes)

Con el objetivo de garantizar el realismo de la simulación, resulta imperativo parametrizar las características físicas y las dinámicas de comportamiento de la flota vehicular. En SUMO, esta configuración se gestiona a través del elemento `vType`, el cual permite especificar variables como la longitud del vehículo, su capacidad de aceleración, velocidad máxima y el modelo de seguimiento vehicular (*car-following model*), https://sumo.dlr.de/docs/Definition_of_Vehicles%2C_Vehicle_Types%2C_and_Routes.html.

Estas definiciones se consolidan en el archivo de configuración adicional `vtype.add.xml`. Para que el simulador reconozca estas definiciones, dicho archivo debe ser referenciado dentro del bloque `<input>` del archivo maestro de configuración `simulation.sumocfg`, bajo la etiqueta `additional-files`.

A continuación vemos un ejemplo de un archivo `vtype.add.xml`:

```
<?xml version='1.0' encoding='utf-8'?>

<additional>

  <vType id="chevrolet_aveo" accel="2.8" decel="4.5" sigma="0.5" length="4.3"
    " minGap="2.5" carFollowModel="Krauss" vClass="passenger" mass="1160"
    personCapacity="5" maxSpeed="55.55" color="#FB0101" emissionClass="
    HBEFA4/PC_petrol_Euro-3" />

  <vType id="chevrolet_sail" accel="2.7" decel="4.5" sigma="0.5" length="4.3"
    " minGap="2.5" carFollowModel="Krauss" vClass="passenger" guiShape="
    passenger/sedan" mass="1090" personCapacity="5" maxSpeed="55.55" color=
    "#FFFF00" emissionClass="HBEFA4/PC_petrol_Euro-4" />

</additional>
```

Listing 3.2: Definición de vTypes en SUMO

A continuación vemos un ejemplo del archivo de configuración de la simulación `sumocfg` que incluye el archivo de vehículos `vtypes.add.xml`:

```
<?xml version='1.0' encoding='utf-8'?>
<input>

  <net-file value="network.net.xml" />
  <route-files value="low.rou.xml" />
  <additional-files value="vtypes.add.xml" />

</input>
<processing>
  .....
</processing>
```

Listing 3.3: Definición de vTypes en SUMO

Objetivos Introducción Instrucciones

Parte I: Generar archivo de salida (outputs) en formato .csv
Parte II: Generar gráficas generales de visualización.
Parte III: Generar gráficas usando herramientas de visualización de SUMO.
Trayectorias:
Velocidad:
Densidad de vehículos:
Emisiones de gases:

4. Estadísticas de movilidad vehicular en SUMO

En esta práctica el estudiante explorará como extraer estadísticas de las simulaciones en SUMO **Modalidad:** Trabajo Individual. **Recursos:** Computador personal con sistema operativo Linux y la SUMO pre-instalado.

4.1 Objetivos

- Generar archivos de salida (outputs) de la simulación en SUMO.
- Entender el contenido de los archivos de salida de las simulaciones.
- Analizar los datos de las simulaciones con scripts en Python.

4.2 Introducción

SUMO permite generar una gran cantidad de estadísticas, basada en los datos de vehículos, simulación, carreteras, intersecciones, semáforos, entre otros. Los archivos de salida escritos por SUMO están en formato XML. SUMO incluye la herramienta `xml2csv.py` que permite convertir los outputs a un formato de archivo plano (csv) que se puede procesar con la mayoría de programas para análisis de datos (R, Python, Excel, etc).

En esta práctica nos enfocamos en 3 ejemplos (outputs) que capturan datos de la red vehicular:

1. **tripinfo**: Esta salida contiene la información sobre el tiempo de viaje del vehículo, distancia recorrida, numero de cambios de rutas entre otros datos. Nota: La información se genera para cada vehículo tan pronto como el vehículo llega a su destino y se retira de la red. En el caso que el vehículo no llegue a su destino durante el tiempo de simulación, este vehículo no se considera en la estadística <https://sumo.dlr.de/docs/Simulation/Output/TripInfo.html>.
2. **fcd**: La exportación FCD (datos de automóviles flotantes) contiene la ubicación, la velocidad, y tipo de vehículo para cada vehículo en la red. Esta información se captura para paso de la simulación (cada 1s por defecto). Esta captura se comporta como un dispositivo GPS de alta frecuencia súper preciso para cada vehículo. Los resultados se pueden procesar aún más utilizando la herramienta TraceExporter para adaptar la frecuencia, las velocidades del equipo, la precisión y el formato de los datos <https://sumo.dlr.de/docs/Simulation/Output/FCDOutput.html>.

3. **trajectories**: Los datos de trayectoria incluyen el nombre, la posición, la velocidad y la aceleración de cada vehículo <https://sumo.dlr.de/docs/Simulation/Output/AmitranOutput.html>.

4.3 Instrucciones

Dividimos la práctica en tres partes:

1. **Parte I**: Generar archivo de salida (outputs) en formato .csv
2. **Parte II**: Generar gráficas generales de visualización.
3. **Parte III**: Generar gráficas usando herramientas de visualización de SUMO.

4.3.1 Parte I: Generar archivo de salida (outputs) en formato .csv

A continuación se describen los pasos para configurar los archivos de salida en una simulación.

1. El primer paso es generar los archivos necesarios para la simulación. Esto se logra de manera manual con las herramientas de SUMO (Práctica 3), o utilizando la herramienta *osmWebWizard* (Práctica 4).
2. Luego, nos enfocamos en modificar el archivo de configuración del escenario `sumo.cfg` o `.sumocfg`. En este archivo XML, es necesario agregar un nuevo bloque con la etiqueta `<outputs>`. En este bloque agregamos las salidas que se requieren capturar, como sigue:

```
<output>
  <tripinfo -output value="tripinfo.xml"/>
  <fcd-output value="fcd.xml">
  <amitran-output value="trajectories.xml">
</output>
```

Dentro del bloque `<outputs>` se indica tres salidas: *tripinfo*, *fcd* y *amitran* outputs. Además, se especifica el nombre del archivo de salida y la ruta donde se guardara el archivo de salida. Cabe anotar que la documentación no presenta la sintaxis de los tópicos de salida. Para conocer la sintaxis, podemos verificar las posibles opciones dentro del tópico `<outputTopicType>` localizado en el archivo `/opt/sumo/data/xsd/sumoConfiguration.xsd`.

3. Una vez agregados los outputs, podemos ejecutar la simulación mediante la CLI o GUI de SUMO. Por ejemplo mediante la CLI, abrimos una terminal y ejecutamos:

```
sumo -c osm.sumocfg
```

4. Una vez finalizada la simulación, de acuerdo a las rutas especificadas para los archivos de salida, podemos encontrarlos en dicha ubicación. Por ejemplo, la Fig. 4.1 se muestra el archivo `tripinfo.xml` generado una vez finaliza la simulación. Dado que este archivo mantiene un formato xml, en el siguiente paso se convierte a .csv para facilitar el manejo de los datos.
5. Para convertir archivos xml en archivos csv, localizamos la herramienta `xml2csv.py`. Esta se ubica en la ruta `/opt/sumo/tools/xml/`. Para ejecutar el script `xml2csv.py`, hemos de indicar el archivo de entrada con la opción `-s`, por ejemplo el archivo `tripinfo.csv`, y la opción `-o` nos permite indicar el archivo de salida (`xxxx.csv`).

```
python3 /opt/sumo/tools/xml/xml2csv.py -s tripinfo.xml, -o tripinfo.csv
```



Figure 4.1: Archivos tripinfo.xml.

4.3.2 Parte II: Generar gráficas generales de visualización.

En este punto, con el archivo .csv se pueden hacer un análisis de los datos o generar gráficas de las estadísticas que se requiera.

Por último, presentamos un script de ejemplo en Python que permite graficar los datos presentes en el archivo tripinfo.csv. Mediante línea de comandos el usuario deba ingresar el path al archivo .csv más las columnas que desea graficar. El nombre de la columna seleccionada debe coincidir con el nombre de la columna en el archivo .csv.

El código ejemplo está disponible de manera pública en el Github <https://github.com/erickperez9/ManualSUMO.git> y fue realizado por el estudiante. Erick Perez. El código se encuentra dentro en ManualSUMO/Scripts plots /plots-v1-4.py.

Por ejemplo, para graficar la columna tripinfo_routeLength, que representan la distancia recorrida, ejecutamos:

```
python3 plots-v1-4.py -i tripinfo.csv --columnas tripinfo_routeLength
```

O si deseamos incluir más columnas en la gráfica podemos agregar emissions_CO2_abs y tripinfo_duration, siendo las emisiones de CO2 y la duración de cada viaje respectivamente.

```
python3 plots-v1-4.py -i tripinfo.csv --columnas tripinfo_routeLength ,
emissions_CO2_abs , tripinfo_duration
```

La Fig. 4.2 presenta un ejemplo del plot generado por el script plots-v1-4.py. La Fig. 4.2 ilustra los resultados del último comando.

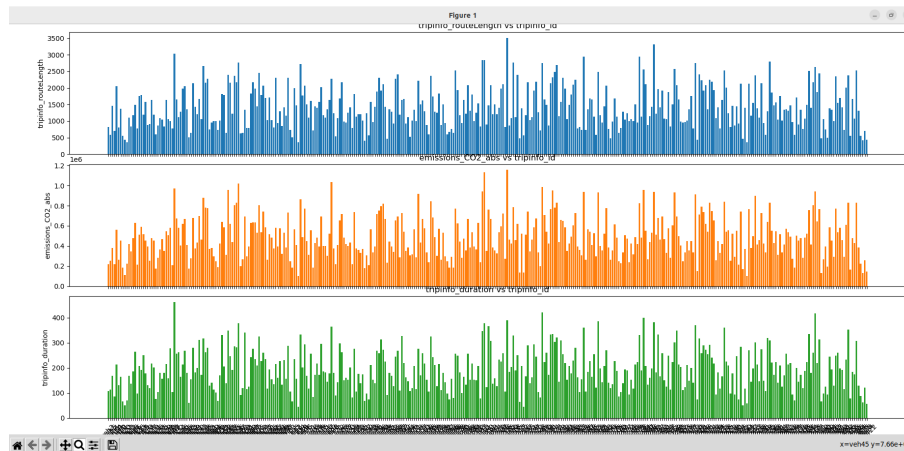


Figure 4.2: Gráficas de: tripinfo_routeLength, emissions_CO2_abs y tripinfo_duration vs tripinfo_id.

4.3.3 Parte III: Generar gráficas usando herramientas de visualización de SUMO.

1. Adicional a los ejemplos de gráficas personalizadas de la Parte II, SUMO incluye diferentes herramientas de visualización de los datos <https://sumo.dlr.de/docs/Tools/Visualization.html>. En esta parte final de la práctica usamos las herramientas de visualización para generar plots para una inspección inicial de la simulación:
 - Trayectorias de vehículos.
 - Velocidad de vehículos.
 - Densidad de vehículos.
 - Emisiones de vehículos.

4.3.4 Trayectorias:

Para graficar las trayectorias de los vehículos de la simulación en SUMO, se puede utilizar dos scripts. El primero es `plotXMLAttributes.py` que está ubicado en la ruta `tools/visualization/`. Para correr este script se requiere como entrada el archivo de salida `fcd_output` generado en la Parte I de la práctica. Ejecutamos:

```
python3 tools/visualization/plotXMLAttributes.py -x x -y y -s 1 -o
allXY_output.png fcd.xml --scatterplot
```

Donde `-x` es el atributo del eje x; `-y` es el atributo del eje y; `-s` es el tamaño; `-o` es el nombre del archivo de salida; `--scatterplot` genera un diagrama de dispersión en lugar de un diagrama de líneas.

La segunda forma para graficar trayectorias es utilizar el script `plot_trajectories.py` que está ubicado en la ruta `tools/`. Para correr este script se requiere como entrada el archivo `fcd_output`. Ejecutamos:


```
python3 tools/plot_trajectories.py -t xy -o allLocations_output.png fcd.xml
```

Donde `-t` es el tipo de trayectoria antes mencionado; `-o` es el nombre del archivo de salida. Además, es posible seleccionar y graficar la trayectoria de uno o más vehículos. Para esto se requiere de aplicar un filtro al script `plot_trajectories.py`, con el parámetro `-filter-ids` el cual indica el ID del vehículo seleccionado. Si se requiere filtrar un vehículo adicional, basta con colocar el ID del otro vehículo seguido de una coma.

La Fig. 4.3 muestra un ejemplo de la gráfica de trayectorias de vehículos. Cada trayectoria se muestra en un color diferente. Vemos que las trayectorias describen la red de carreteras urbanas.

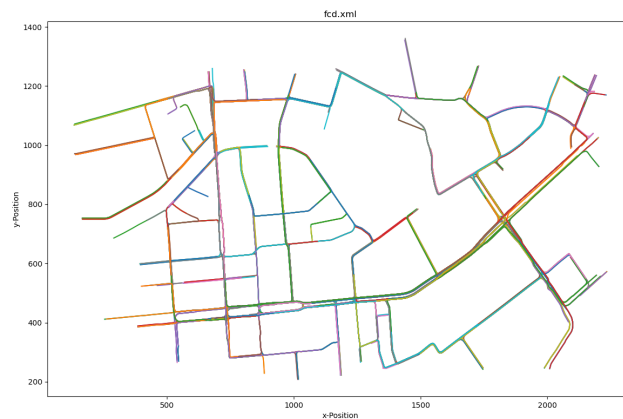


Figure 4.3: Trayectoria de vehículos en la simulación. Gráfica generada con herramientas de SUMO.

4.3.5 Velocidad:

La gráfica de velocidades de vehículos nos permite comparar la velocidad de los vehículos durante la simulación. Usamos el script `plot_trajectories.py` ubicado en `/tools`. El script requiere como entrada el archivo `fcd.xml`. A continuación se muestra un ejemplo de uso:

```
python tools/plot_trajectories.py -t ts -o timeSpeed_output.png fcd.xml
--filter-ids veh1, veh40
```

En la Figura 4.4 se muestra las velocidades de los vehículo con IDs *1* y *40*. Se observa que el vehículo *1* comienza su viaje en es segundo 0 y termina en el segundo 220, con sus variaciones respectivas de velocidad en *m/s*. Mientras que el vehículo *40*, comienza su recorrido en el segundo 320 y termina en el segundo 500.

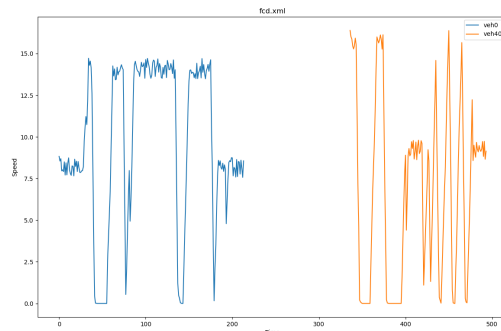


Figure 4.4: Comparación de velocidades de los vehículos *veh1* y *veh40*.

4.3.6 Densidad de vehículos:

Otra de las herramientas que incluye SUMO es `plot_net_dump.py` ubicado en `tools/visualization/`. Este script muestra una red, donde los colores y el ancho de los bordes de la red se establecen en función de los datos en las carreteras.

Las medidas de densidad se localizan en el archivo `edge.data.xml` (https://sumo.dlr.de/docs/Simulation/Output/Lane_or_Edge-based_Traffic_Measures.html). Este archivo contiene datos de cada carretera como la ocupación o densidad (veh/km) en cada instante de la simulación.

Note que no hemos generado este output anteriormente. Bastaría con agregar el tópico respectivo (`edgedata-output`) en los outputs del archivo `sumo.cfg`

```
<output>
  <emission-output value="co2.xml"/>
  <tripinfo-output value="tripinfo.xml"/>
  <fcd-output value="fcd.xml"/>
  <edgedata-output value="edgedata.xml"/>
</output>
```

El archivo resultante `edgedata.xml` almacena entre otros datos la densidad de vehículos en cada carretera *veh/km*. A continuación se muestra un ejemplo de uso de esta herramienta para graficar la densidad de los vehículos en cada carretera:

```
python3 /home/sumo/sumo/tools/visualization/plot_net_dump.py -v -n osm.
net.xml --measures density,density --xlabel [m] --ylabel [m] --
default-width 1 -i edgedata.xml,edgedata.xml --xlim 000,2600 --ylim
000,1500 --default-width .5 --min-color-value 0 --max-color-value 5 --
max-width-value 5 --min-width-value 0 --max-width 3 --min-width .5 --
colormap winter -o density-color-map.png
```

Donde:

- `-v` (verbose) imprime el progreso en pantalla.
- `-n` indica el archivo de entrada de red, es decir, el archivo `.net`.
- `-xlabel [m]` indica la etiqueta del eje x.

- `-ylabel [m]` indica la etiqueta del eje y.
- `-default-width` indica el valor del ancho de las línea de las carreteras.
- `-i` indica los archivos de entrada donde se encuentra la medida a graficar.
- `-xlim` indica el límite del mapa en metros del eje x.
- `-ylim` indica el límite del mapa en metros del eje y.
- `-min-color-value` define el valor mínimo de color del borde
- `-max-color-value` define el valor máximo de color del borde
- `-min-width-value` define el valor del ancho mínimo del borde.
- `-max-width` define el ancho máximo del borde.
- `-min-width` define el ancho mínimo del borde.
- `-colormap` indica el color de la barra de valores.
- `-o` indica el archivo de salida.

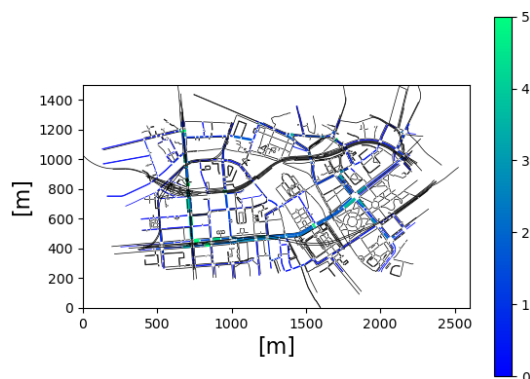


Figure 4.5: Mapa de color de densidad de flujo de vehículos.

La Fig. 4.5, muestra la densidad por carreta. Por medio de la barra de calor a la derecha, vemos sobre el mapa las carreteras con mayor (verde) y menor (azul) cantidad de vehículos durante la simulación. En este ejemplo graficamos la densidad en las carreteras, por tal motivo se indica el valor `-measures` como `density,density`; sin embargo podemos usar otras capturas como la ocupación (https://sumo.dlr.de/docs/Simulation/Output/Lane_or_Edge-based_Traffic_Measures.html).

4.3.7 Emisiones de gases:

Para lograr esto, se requiere generar los datos de emisiones de los vehículos en la simulación. A diferencia de los ejemplos anteriores donde se agregaba el tópico en el archivo de configuración del escenario `sumo.cfg`, aquí se requiere (i) agregar la configuración en un archivo adicional (`additional-file`) y (ii) importar el archivo adicional dentro del bloque `inputs` del `sumocfg`.

(i) El nuevo archivo es el `edgeData` https://sumo.dlr.de/docs/Simulation/Output/Lane_or_Edge-based_Emissions_Measures.html. Donde, se debe especificar el `id`, el periodo (tiempo total de simulación) de toma de datos, el tipo de datos a recibir (emisiones)

y el nombre del archivo de salida. A continuación se muestra el código de este nuevo archivo.

```
<additional xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:
  noNamespaceSchemaLocation="http://sumo.dlr.de/xsd/additional_file.xsd"
  ">

  <edgeData id="1" period="3600" type="emissions" file="edgeData.
    xml" excludeEmpty="true"/>

</additional>
```

(ii) Para importar el nuevo archivo creado al archivo de configuración de sumo se requiere indicar la ruta de ubicación de este archivo en el apartado de archivos adicionales. Por ejemplo:

```
<input>
  <net-file value="osm.net.xml.gz"/>
  <route-files value="osm.passenger.trips.xml"/>
  <additional-files value="osm.poly.xml.gz,edgeData.xml"/>
</input>
```

La Fig. 4.6, muestra las emisiones absolutas (total) de CO₂ en cada carretera del mapa. Usamos la herramienta `plot_net_dump.py` para graficar el mapa de color en base a las emisiones CO₂ medidas en las carreteras del mapa. Note que, adicional a los nombres del archivo de entrada y salida, el valor a graficar también cambia (`-measures CO2_abs,CO2_abs`), así como los límites respectivos.

```
python3 /home/sumo/sumo/tools/visualization/plot_net_dump.py -v -n osm.
net.xml --measures CO2_abs,CO2_abs --xlabel [m] --ylabel [m] --
default-width 1 -i edgeData-output-v2.xml,edgeData-output-v2.xml --
xlim 000,2600 --ylim 000,1500 --default-width .2 --min-color-value 0
--max-color-value 2000000 --max-width-value 2000000 --min-width-value
0 --max-width 1 --min-width 1 --colormap turbo -o density-color-map.
png
```

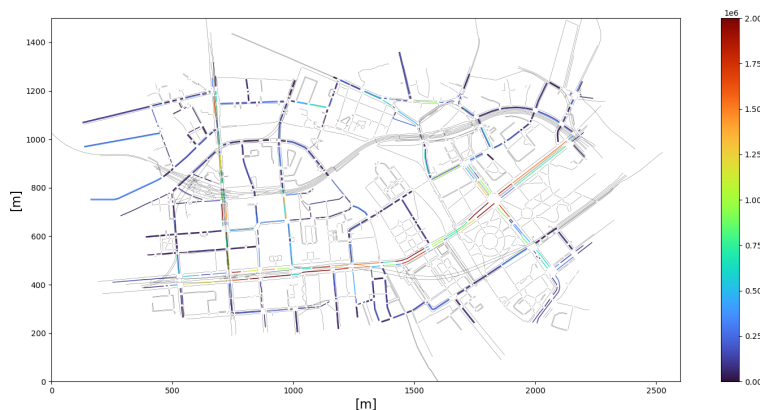


Figure 4.6: Mapa de color de CO₂ absoluto de vehículos.

Objetivos Introducción Desarrollo

Parte I: Instalar librería TraCI para Python.
Parte II: Ejecutar una simulación desde Python usando la interfaz TraCI
Parte III: Ejemplos de interacción TraCI - SUMO.

Reto TraCI

5. Introducción a TraCI (Traffic Control Interface)

En esta práctica, el estudiante conocerá como controlar una simulación de tráfico en tiempo real a través de la interfaz TraCI (Traffic control interface). Se utiliza el lenguaje Python para generar una simulación bidireccional y dinámica. Se proporcionan códigos base como punto de partida para el estudiante.

5.1 Objetivos

- Conocer la librería TraCI para el control de tráfico.
- Integrar la interfaz TraCI con SUMO.
- Ejecutar una simulación vehicular desde Python utilizando TraCI.

5.2 Introducción

TraCI es el término corto para "Interfaz de control de tráfico" (Traffic Control Interface - Documentación https://sumo.dlr.de/docs/TraCI/Interfacing_TraCI_from_Python.html). TraCI permite controlar una simulación de tráfico en marcha, recuperar valores de objetos simulados y manipular su comportamiento en línea. Esta interfaz es de especial interés cuando necesitamos interactuar en línea con los objetos de la simulación, por ejemplo, cambiar la ruta de un vehículo o los ciclos semafóricos en función del estado del tráfico (i.e., control adaptativo).

TraCI utiliza una arquitectura cliente/servidor basada en TCP para proporcionar acceso a SUMO. De este modo, SUMO actúa como un servidor que se inicia con opciones adicionales de línea de comandos: `-remote-port <INT>`, donde `<INT>` es el puerto en el que sumo escuchará las conexiones entrantes. SUMO carga los archivos de simulación y espera a que las aplicaciones externas se conecten y tomen el control. Un elemento a tener en cuenta es que la opción `-end <TIME>`, definida en el `sumo.cfg`, se ignora cuando SUMO se ejecuta como servidor TraCI; SUMO se ejecuta hasta que el cliente (script en Python) exige el fin de la simulación.

En la Fig. 5.1, después de iniciar el servidor SUMO, los clientes se conectan a SUMO configurando una conexión TCP al puerto de sumo designado. La aplicación cliente envía comandos a sumo para controlar la ejecución de la simulación, influir en el comportamiento de un solo vehículo o solicitar detalles ambientales. SUMO responde con una respuesta de estado a cada comando y resultados adicionales que dependen del comando dado.

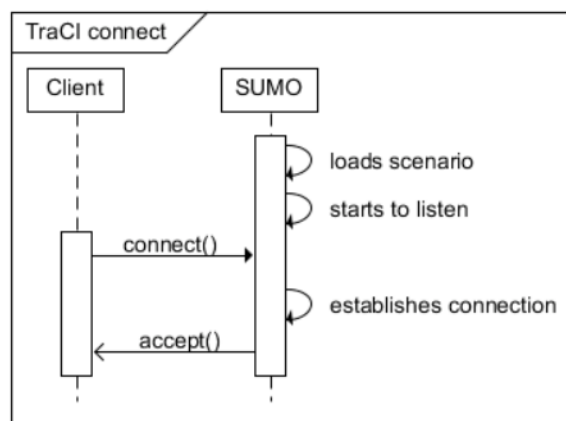


Figure 5.1: Inicio de conexión.

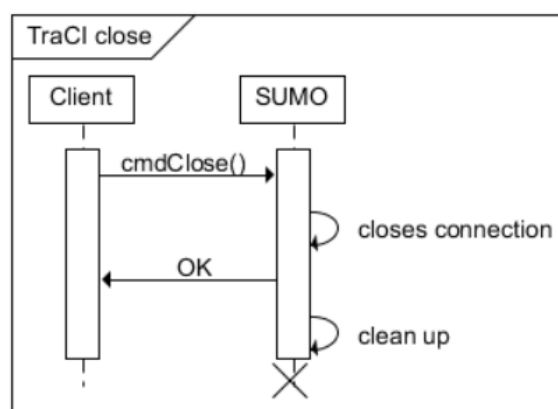


Figure 5.2: Fin de conexión.

Table 5.1: Capacidades y Operaciones de la Interfaz TraCI en SUMO.

Categoría	Acción / Operación	Elementos Involucrados
Recuperación de Valores	Consulta de datos en tiempo real (get)	Vehículos, Detectores, Semáforos, Edges/Lanes.
Cambio de Estado	Modificación de parámetros (set)	Ruta de vehículos, programas de semáforos, velocidad.
Suscripciones	<i>Variable Subscription</i>	Monitoreo automático de variables específicas de un objeto.
Suscripciones	<i>Context Subscription</i>	Monitoreo de objetos dentro de un radio de proximidad.

En la Fig. 5.2, el cliente es responsable de cerrar la conexión mediante el comando de cierre. Cuando todos los clientes emitidos cierran, la simulación finalizará, liberando todos los recursos.

Mediante la interfaz TraCI, podemos: recurrir valores, cambiar el estado o suscribirnos a diferentes elementos de la simulación (ver Tabla 5.1).

La Fig. 5.3, muestra la librería en tres pilares fundamentales que permiten el control externo de una simulación en SUMO:

- **Funciones (Métodos de Control):** Son los comandos de alto nivel para gestionar la simulación. Incluye métodos críticos como `start()` para iniciar SUMO, `simulationStep()` para avanzar el tiempo de simulación y `close()` para finalizar la conexión.
- **Módulos (Estructura Interna):** Son las bibliotecas internas y externas que TraCI utiliza para funcionar (como `socket` para la comunicación TCP/IP o `traci.vehicle` para acceder a los métodos específicos de autos).
- **Datos (Dominios de Objetos):** Representan los objetos del simulador tratados como clases de Python (instancias). Por ejemplo, `vehicle` es un objeto de tipo `VehicleDomain` que contiene todos los métodos para consultar o modificar vehículos.

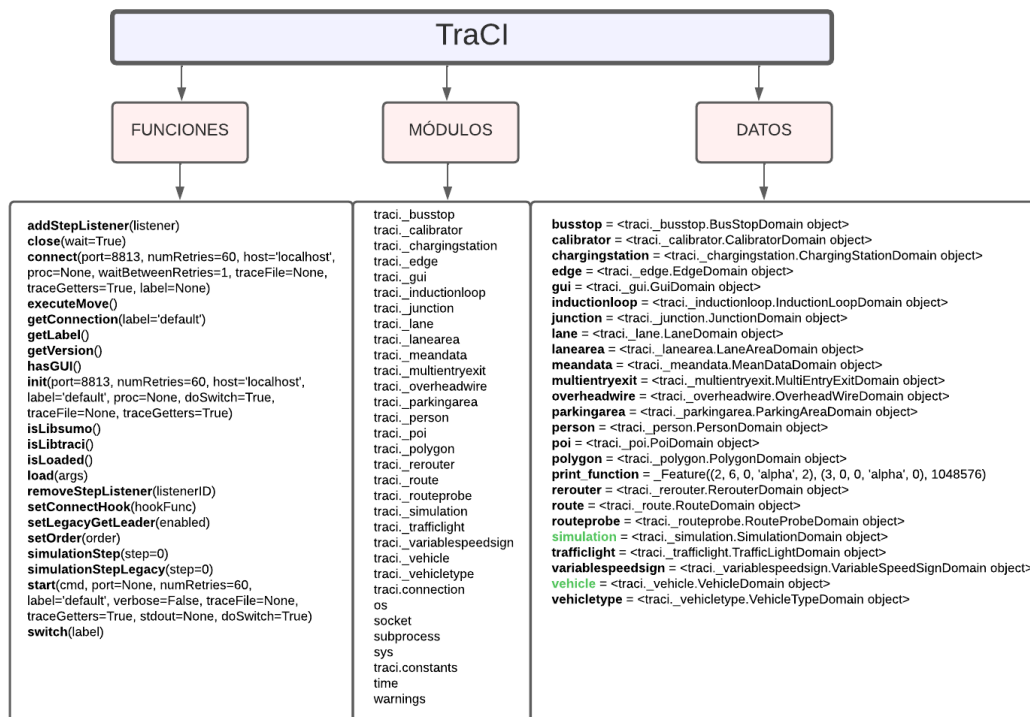


Figure 5.3: Diagrama de funcionamiento de TraCI.py.

5.3 Desarrollo

El objetivo principal de esta práctica es la orquestación y el control bidireccional de un entorno de movilidad mediante la ejecución de scripts en Python, utilizando la interfaz TraCI para interactuar en tiempo real con el simulador SUMO. Esta arquitectura permite la gestión dinámica de eventos imprevistos; por ejemplo, ante la detección de un incidente vial (accidente), el sistema puede intervenir sobre el dominio de Datos (específicamente sobre el objeto vehicle) para reencaminar de manera automática a los agentes cuya trayectoria original incluya el segmento afectado.

Valide con el docente

El desarrollo de la práctica se divide en tres etapas secuenciales. Una vez completadas, es obligatoria la validación de los resultados con el docente. **En el informe deberá reportar únicamente el desarrollo de la Parte IV.**

1. **Parte I:** Instalar TraCI para Python.
2. **Parte II:** Ejecutar una simulación desde Python usando la interfaz TraCI.
3. **Parte III:** Ejemplos de interacción TraCI - SUMO.
4. **Parte IV:** Reto TraCI

5.3.1 Parte I: Instalar librería TraCI para Python.

1. Verifique que Python 3 esté instalado en el sistema. Desde una consola, escriba `python3 --version` y retorne la versión instalada.
2. Para utilizar la librería TraCI en Python 3, puede instalarla mediante el gestor de paquetes ejecutando el comando `pip3 install traci`, o bien, realizar la instalación directamente desde su Entorno de Desarrollo Integrado (IDE) de preferencia. Tenga en cuenta que, si emplea un entorno virtual (`venv`) dentro de su IDE, es fundamental verificar la configuración de las variables de entorno (como `SUMO_HOME`) para garantizar que el sistema reconozca los comandos de SUMO correctamente
3. En este punto, ya es posible conectar TraCI con SUMO e implementar la lógica para el control dinámico de la simulación la codificación para requerimientos específicos, por ejemplo, cambiar la ruta de un vehículo dado durante la simulación.

5.3.2 Parte II: Ejecutar una simulación desde Python usando la interfaz TraCI

Dentro del ecosistema de herramientas de SUMO, TraCI desempeña un papel crucial al permitir la comunicación en tiempo real entre simulaciones externas y el simulador SUMO. De especial interés cuando trabajamos con programas externos como simuladores de redes vehiculares (e.j., Veins) o para aplicaciones de ML. TraCI permite controlar y monitorear de manera dinámica los elementos de la simulación, como vehículos, semáforos, y otros agentes de tráfico.

En los siguientes pasos, describimos los elementos básicos de un script en Python que utiliza la interfaz TraCI para controlar una simulación de SUMO. Para facilidad, se proporciona plantillas de código disponibles en el repositorio público de Github: <https://github.com/erickperez9/ManualSUMO.git>. Se recomienda en este punto clonar el repositorio para los siguientes pasos:

1. Primero, vamos a verificar que la variable de entorno `$SUMO_HOME` esté configurada correctamente. Utilice el script `Scripts TraCI_SUMO/traci_check_sumo_home.py` disponible en el Github. Como resultado, se muestra el path donde se instaló la librería `traci`.
2. A continuación presentamos el código base en Python para ejecutar una simulación vehicular con TraCI. Disponible en la ubicación `Scripts TraCI_SUMO/traci_run_sumo.py` del Github.

```
import traci

traci.start([ "sumo", "-c", "/home/simulation.sumocfg" ])

while traci.simulation.getMinExpectedNumber() > 0:
    traci.simulationStep()
traci.close()
```

El primer paso es importar las librerías necesarias. Como ejemplo solo se usa `traci`. Luego, es necesario iniciar la comunicación SUMO/TraCI a través del comando `traci.start([])`. Para el ejemplo usamos la CLI de SUMO, pero también se podría usar la GUI `sumo-gui`.

```

Simulation ended at time: 3950.00
Reason: TraCI requested termination.
Performance:
  Duration: 9.26s
  TraCI-Duration: 0.70s
  Real time factor: 426.796
  UPS: 8284.062669
Vehicles:
  Inserted: 389
  Running: 0
  Waiting: 0
Statistics (avg of 389):
  RouteLength: 1446.19
  Speed: 7.45
  Duration: 197.09
  WaitingTime: 28.26
  TimeLoss: 60.80
  DepartDelay: 0.58

DijkstraRouter answered 389 queries and explored 354.39 edges on average.
DijkstraRouter spent 0.08s answering queries (0.19ms on average).
sumo@sumo-VirtualBox:~/sumo/tools/2024-02-05-16-21-35$

```

Figure 5.4: Resumen de la simulación TraCI/SUMO en consola usando TraCI.

Luego, el parámetro `-c` indica la ubicación del archivo de configuración de la simulación de SUMO `simulation.sumocfg`.

Dado que la simulación es iterativa, es necesario iniciar un bucle donde la condición lógica indica la duración que deseamos para el escenario simulado (se omite lo configurado en `end value` del `sumocfg`). Esta condición lógica pueden ser el número de pasos de la simulación, velocidades, ubicaciones, etc. Para el ejemplo, tomamos el valor del número mínimo esperado de vehículos en la simulación (`getMinExpectedNumber()`), lo que significa que la simulación se realizará mientras existan vehículos en la simulación. Esto depende del archivo de rutas `.rou.xml`. Dentro del bucle, la función `simulaciónStep()` dentro del bucle realiza un paso de simulación en cada iteración. Si el valor dado es 0 o está ausente, se realiza exactamente un paso (definido en el `sumocfg`). En `.sumocfg`, el parámetro `step-length` define la granularidad de la física (qué tan "fino" es el cálculo de colisiones y movimientos). En Python, la función `simulationStep()` es el disparador lógico que le dice al servidor "calcula el siguiente estado".

Por último, es importante **cerrar la comunicación con el servidor**, caso contrario tendremos instancias corriendo en background y puertos ocupados.

3. Como resultado de la simulación, en la Fig. 5.4, observamos un resumen de la simulación en consola y algunas estadísticas.
4. Otro ejemplo común para ejecutar el bucle de simulación, es considerar los pasos que tiene la simulación. Estos pasos están relacionados con el tiempo de simulación, pero no necesariamente es el mismo configurado en el archivo de configuración del escenario (`sumocfg`). A continuación vemos el script de Python correspondiente a la lógica del bucle considerando un tiempo estimado de simulación de una hora (3600 segundos) y 3700 pasos:

```

import traci

traci.start([ "sumo", "-c", "/home/osm.sumocfg" ])
step = 0

```

```
while step < 3700:
    traci.simulationStep()
    step += 1
traci.close()
```

5.3.3 Parte III: Ejemplos de interacción TraCI - SUMO.

En esta última parte de la práctica presentamos cuatro ejemplos de interacción entre TraCI y SUMO. Los código de ejemplo se puede descargar o clonar del repositorio Github <https://github.com/erickperez9/ManualSUMO.git>. Cabe mencionar que los códigos presentados son referenciales, el estudiante puede usarlos como punto de partida para sus experimentos aplicando su lógica particular. Presentamos 4 ejemplos:

1. Conteo de vehículos en línea.
2. Cálculo de velocidad promedio de vehículos en línea.
3. Cálculo de emisiones CO2 de vehículos en línea.
4. Cambio de trayectoria de vehículos en línea.

Ejemplo 1: Conteo de vehículos en línea.

- **Ubicación Github:** el ejemplo de conteo de vehículos se encuentra en: Scripts TraCI_SUMO/traci_cont_veh.py.
- **Importando librerías (Líneas 1, 4):** se utilizan `traci` para la conexión con SUMO, y `re` para el manejo de expresiones regulares.
- **Iniciamos la conexión SUMO/TraCI (Línea 8):** a través del comando `traci.start(["cmd"])`. Note que debe modificar el path donde se encuentra su propio archivo de configuración del escenario (`sumo.cfg`).
- **Bucle de simulación (Líneas 14-20):** usamos `traci.vehicle.getIDList()` (Línea 15) para obtener una lista de todos los IDs de vehículos presentes en la simulación en el paso actual. El comando devuelve los IDs en el formato `vehID`, donde `ID` indica el número de vehículo de la simulación. En la línea 20, el comando `traci.simulationStep()` permite avanzar un paso en la simulación.
- **Cierre de la comunicación (Línea 22):** una vez terminada la simulación, es importante cerrar el socket con SUMO.
- **Conteo de los IDs de los vehículos (Líneas 24-38):** es necesario crear una lógica con la cual se extraiga los números identificadores de cada vehículo (e.j., `veh15`) y obtener el número mayor.

Ejemplo 2: Cálculo de velocidad promedio de vehículos en línea.

1. **Ubicación Github:** el ejemplo de cálculo de la velocidad de vehículos se encuentra en: Scripts TraCI_SUMO/traci_vel.py.
2. **Importando librerías (Líneas 1, 7):** En este caso se utilizará `traci` para la conexión con SUMO. Además de, `csv` para el manejo de archivos csv, `pandas` para análisis de datos, `time` para obtener los tiempos de simulación, `matplotlib` para generar gráficas.
3. **Iniciamos la conexión SUMO/TraCI (Línea 8):** a través del comando `traci.start(["cmd"])`.

Note que debe modificar el path donde se encuentra su propio archivo de configuración del escenario (sumo.cfg).

4. **Preparamos archivo csv para almacenar datos (Líneas 15-21):** creamos y etiquetamos las columnas del archivo csv.
5. **Bucle de simulación (Líneas 24-33):** guardamos los datos de interés: identificadores de los vehículos, velocidades, y tiempos de simulación. Los datos se guardan en un archivo de salida de tipo csv.
6. Para obtener el tiempo de simulación en segundos usamos (Línea 25):
`traci.simulation.getCurrentTime() / 1000.`
7. Para obtener los IDs usamos (Línea 27). Note que generamos un bucle adicional para iterar sobre cada vehículo: `traci.vehicle.getIDList()`.
8. Para obtener las velocidad de cada vehículo, usamos (Línea 28):
`traci.vehicle.getSpeed(veh_id).`
9. **Cierre de la comunicación (Línea 35):** una vez terminada la simulación, es importante cerrar el socket con SUMO.
10. **Análisis de datos (Líneas 37-48):** En el paso anterior creamos un archivo .csv. Ahora leemos el archivo con los datos de velocidad de cada vehículo en cada instante de tiempo y calculamos la velocidad media por vehículo y de todos los vehículos (C.I. 95%).

Ejemplo 3: Cálculo de emisiones CO2 de vehículos en línea.

1. **Ubicación Github:** el ejemplo de cálculo de emisiones de gases se encuentra en:
`Scripts TraCI_SUMO/traci_co2.py.`
2. **Importando librerías (Líneas 1 , 7):** En este caso se utilizará `traci` para la conexión con SUMO. Además de, `csv` para el manejo de archivos csv, `pandas` para análisis de datos, `time` para obtener los tiempos de simulación, `matplotlib` para generar gráficas y `seaborn`, `stats` para estadísticas y gráficas.
3. **Iniciamos la conexión SUMO/TraCI (Línea 10):** a través del comando `traci.start(["cmd"])`. Note que debe modificar el path donde se encuentra su propio archivo de configuración del escenario (sumo.cfg).
4. **Preparamos archivo csv para almacenar datos (Líneas 13-20):** creamos y etiquetamos las columnas del archivo csv.
5. **Bucle de simulación (Líneas 15-32):** guardamos los datos de interés: identificadores de los vehículos, emisiones de CO2, y tiempos de simulación. Los datos se guardan en un archivo de salida de tipo csv.
6. Para obtener el tiempo de simulación en segundos se usa:
`traci.simulation.getCurrentTime() / 1000`
7. Para obtener los IDs se utiliza: `traci.vehicle.getIDList()`
8. Para obtener las emisiones de CO2 se utiliza: `traci.vehicle.getCO2Emission(veh_id)`
9. **Cierre de la comunicación (Línea 35):** una vez terminada la simulación, es importante cerrar el socket con SUMO.
10. **Análisis de datos (Líneas 37-48):** En el paso anterior creamos un archivo .csv. Ahora leemos el archivo con las capturas de emisiones (**mg/s**) y calculamos la media de cada vehículo y de la simulación en general.

Ejemplo 4: Cambio de trayectoria de vehículos en línea.

Para el ejemplo, creamos una lógica sencilla para generar el cambio de ruta en un vehículo. Si la velocidad es menor a 10 m/s, el destino del vehículo con ID *veh424* se modifica y se re-dirige al destino con ID *234463928#1* (ID de la carretera). SUMO utiliza el algoritmo Dijkstra para calcular la nueva ruta al destino.

Los IDs de los bordes de las carreteras se pueden verificar en los archivos de configuración de red pertinentes (.net) o utilizando a herramienta *netedit* de SUMO al dar clic sobre la carretera en cuestión. Para abrir esta herramienta basta con el comando *netedit* desde la terminal (ver Fig. 5.5).

1. **Ubicación Github:** el ejemplo de cambio de trayectoria se encuentra en:
`Scripts TraCI_SUMO/traci_change_pos.py`.
2. **Importando librerías (Líneas 1 - 3):** En este caso se utilizará `traci` para la conexión con SUMO. Además de, `csv` para el manejo de archivos csv.
3. **Iniciamos la conexión SUMO/TraCI (Línea 9):** a través del comando `traci.start(["cmd"])`. Note que debe modificar el path donde se encuentra su propio archivo de configuración del escenario (`sumo.cfg`).
4. **Preparamos archivo csv para almacenar datos (Líneas 11 - 18):** creamos y etiquetamos las columnas del archivo csv. Note que se incluye `EdgeNow` que hace referencia a los IDs de las carreteras en cada instante.
5. **Bucle de simulación (Líneas 21-36):** guardamos los datos de interés: identificadores de los vehículos, tiempo de simulación, IDs de las carreteras, y velocidad. Los datos se guardan en un archivo de salida de tipo csv.
6. Para obtener el tiempo de simulación en segundos se usa:
`traci.simulation.getCurrentTime() / 1000`
7. Para obtener los IDs se utiliza: `traci.vehicle.getIDList()`
8. Para obtener las velocidad se utiliza: `traci.vehicle.getSpeed(veh_id)`
9. Para obtener el ID de la calle (edge) donde se encuentra el vehículo actualmente se utiliza:
`traci.vehicle.getRoadID(veh_id)`
10. **Para cambiar el destino de un vehículos se utiliza:**
`traci.vehicle.changeTarget(veh_id,EDGE_ID)`
11. **Cierre de la comunicación (Línea 38):** una vez terminada la simulación, es importante cerrar el socket con SUMO.

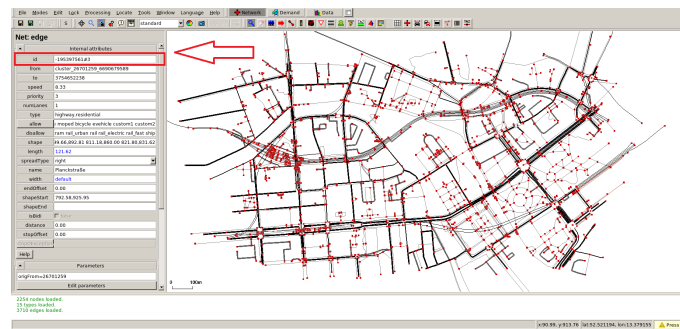


Figure 5.5: Identificador de calle desde netedit, editor gráfico del mapa en SUMO.

5.4 Reto TraCI

Documentar en el informe: El desafío consiste en desarrollar un script de Python que, utilice TraCI, y gestione dinámicamente la prioridad de una ambulancia en una red saturada (tráfico en hora pico). Los estudiantes deberán:

1. **Generar demanda vehicular:** Crear la demanda vehicular en hora pico para el mapa seleccionado. Incluya buses, vehículos privados (passenger) y un vehículo de emergencia (ambulancia).
2. **Control de Infraestructura:** Generar un accidente aleatorio de un vehículo privado (random vehID). En este momento la ambulancia debe dirigirse al lugar del accidente (edge ID).
3. **Priorización Semafórica:** Monitorear la posición y ruta de la ambulancia para detectar la proximidad a las intersecciones controladas por semáforos. Activar el estado "verde" mediante las funciones de control de semáforos al detectar al vehículo de emergencia. https://sumo.dlr.de/docs/Simulation/Traffic_Lights.html
4. **Cooperación Vehicular:** Implementar una lógica que obligue a los conductores circundantes a reducir su velocidad o cambiar de carril para abrir paso.